Proyecto

Máster en Ingeniería de Sistemas y Control

Asignatura: Prácticas de Computación y Robótica

Curso: 2025/2026

Alumno: Javier Arambarri Calvo

5 de marzo de 2026

Autoría

El autor de este documento es Javier Arambarri Calvo, alumno del Máster en Ingeniería de Sistemas y Control y matriculado por la Universidad Complutense de Madrid en el curso académico 2025-2026.

Para que así conste, se firma digitalmente este documento.

Índice

Instalación y despliegue **6**

Tarea 1: experimento de control de posición de un robot móvil. **7**

1. Descargar el simulador y agregar el modelo del robot al mismo como se explica en el documento Interfaz de la Práctica. . . . 7
2. Al arrastrar el robot desde el panel lateral y soltarlo en el espacio de trabajo, el robot quedará agregado a la simulación. . . . 7
3. Iniciar la simulación y observar el comportamiento del robot. ¿A qué se debe este comportamiento? . . . 7
4. Abrir el script de programación del robot y observar los valores de las velocidades izquierda y derecha. . . . 7
5. Agregar a la simulación un objeto que represente el punto de destino del robot. . . . 8
6. Con las coordenadas del punto de destino y las del robot, calcular las velocidades angular y lineal para controlar su posición. . . . 9
7. Obtener las velocidades que se envían al robot (izquierda y derecha) como resultado del modelo diferencial. . . . 9
8. Implementar una ley de control diferente a la proporcionada y comparar ambas usando los Índices de Comportamiento (*IAE*, *ISE*, *ITAE*, *ITSE*). . . . 9

Tarea 2: control de posición con evitación de obstáculos. **25**

1. Agregar obstáculos al espacio de trabajo, pueden ser paredes, cilindros, etc... **nota**: habilitar la propiedad "detectable" de los obstáculos para que el robot los pueda detectar con los sensores. . . . 25
2. Implementar el algoritmo de *Braitenberg* por medio de la manipulación de las velocidades del robot. . . . 25
3. Iniciar la simulación y observar el comportamiento del robot. ¿A qué se debe este comportamiento? . . . 29
4. Abrir el script de programación del robot y observar los valores de las velocidades izquierda y derecha. . . . 29
5. Agregar a la simulación un objeto que represente el punto de destino del robot. . . . 30
6. Con las coordenadas del punto de destino y las del robot, calcular las velocidades angular y lineal para controlar su posición. . . . 30
7. Obtener las velocidades que se envían al robot (izquierda y derecha) como resultado del modelo diferencial. . . . 30
8. Implementar una ley de control diferente a la proporcionada y comparar ambas usando los Índices de Comportamiento (*IAE*, *ISE*, *ITAE*, *ITSE*). . . . 30

ANEXO I: código **47**

Referencias **60**

Índice de figuras

1. Robot *Khepera IV* y objeto que representa el punto de destino agregados a *Coppelia Sim*. . . . 7
2. Script por defecto del robot. . . . 8
3. Propiedades de la esfera añadida como punto de destino. . . . 8
4. Esquema de la arquitectura propuesta. . . . 13
5. Esquema de un robot móvil diferencial con ruedas [**FARIAS20179150**]. . . . 14
6. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}1$. 17
7. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}15$. 17
8. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}2$. 18
9. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}25$. 18
10. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}3$. 18
11. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}35$. 19
12. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}4$. 19
13. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}45$. 19
14. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}5$. 20
15. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}55$. 20
16. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}6$. 20
17. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}1$. . . . 21
18. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}15$. . . . 21
19. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}2$. . . . 21
20. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}25$. . . . 22
21. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}3$. . . . 22
22. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}35$. . . . 22
23. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}4$. . . . 23
24. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}45$. . . . 23
25. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}5$. . . . 23
26. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}55$. . . . 24
27. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}6$. . . . 24
28. Vehículos de *Braitenberg* [**dicaro2017behaviors**]. . . . 25
29. Robot *Khepera IV* y objeto que representa el punto de destino agregados a *Coppelia Sim*. . . . 29
30. Script por defecto del robot. . . . 30
31. Propiedades de la esfera añadida como punto de destino. . . . 30
32. Esquema de la arquitectura propuesta. . . . 34
33. Esquema de un robot móvil diferencial con ruedas [**FARIAS20179150**]. . . . 35
34. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}1$. 39

35. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}15$. 39
36. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}2$. 39
37. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}25$. 40
38. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}3$. 40
39. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}35$. 40
40. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}4$. 41
41. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}45$. 41
42. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}5$. 41
43. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}55$. 42
44. Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}6$. 42
45. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}1$. 42
46. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}15$. 43
47. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}2$. 43
48. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}25$. 43
49. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}3$. 44
50. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}35$. 44
51. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}4$. 44
52. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}45$. 45
53. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}5$. 45
54. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}55$. 45
55. Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}6$. 46

Índice de tablas

1. Evaluación de los controladores con las cuatro métricas propuestas para distintos valores de K_p. 17
2. Comparativa de métodos de implementación de Braitenberg. 27
3. Evaluación de los controladores con las cuatro métricas propuestas para distintos valores de K_p. 38

Instalación y despliegue

Con el objetivo de facilitar la instalación y el despliegue del entorno de desarrollo utilizado, se recomienda utilizar la imagen *Docker* que se ha preparado. Esta imagen incluye tanto el simulador *CoppeliaSim* como el ecocsistema de *ROS2*.

Se puede construir utilizando el fichero *Dockerfile*, Código 1, con el comando "`docker build -t coppelia_sim_ros2:jazzy .`" o descargar mediante el comando: "`docker pull arambarricalvoj/coppelia_sim_ros2:jazzy`".

Una vez descargada la imagen, para crear un contenedor o instancia de la misma ejecutar desde terminal el fichero *run.sh*, Código 2. La terminal accederá al contenedor *Docker* iniciado y arrancará automáticamente el simulador.

Para ejecutar la aplicación programada en *ROS2*, desde otra terminal ejecutar "`docker exec -it coppelia_sim bash`". La aplicación estará en la ruta `/home/$USER/pcr_misc/tareas/ros2_ws`.

Todo este proceso junto con los comandos específicos para ejecutar cada tarea se detalla en más profundidad en el repositorio del proyecto: `https://github.com/arambarricalvoj/pcr_misc_ucm-uned_25-26`.

Tarea 1: experimento de control de posición de un robot móvil.

El experimento consiste en controlar la posición de un robot móvil con el Simulador CoppeliaSim. Los pasos a llevar a cabo son los siguientes:

1. Descargar el simulador y agregar el modelo del robot al mismo como se explica en el documento Interfaz de la Práctica.

Se ha creado una imagen de *Docker* con el simulador instalado, el modelo del robot *Khepera IV*, y el *framework ROS2* [**doi:10.1126/scirobotics.abm6074**]. En el apartado **??** se detalla la descarga e instalación utilizando *Docker*. En la Figura 29 se observa que el robot se ha cargado correctamente.

2. Al arrastrar el robot desde el panel lateral y soltarlo en el espacio de trabajo, el robot quedará agregado a la simulación.

En la Figura 29 se observa que el robot se ha agregado correctamente a la simulación.

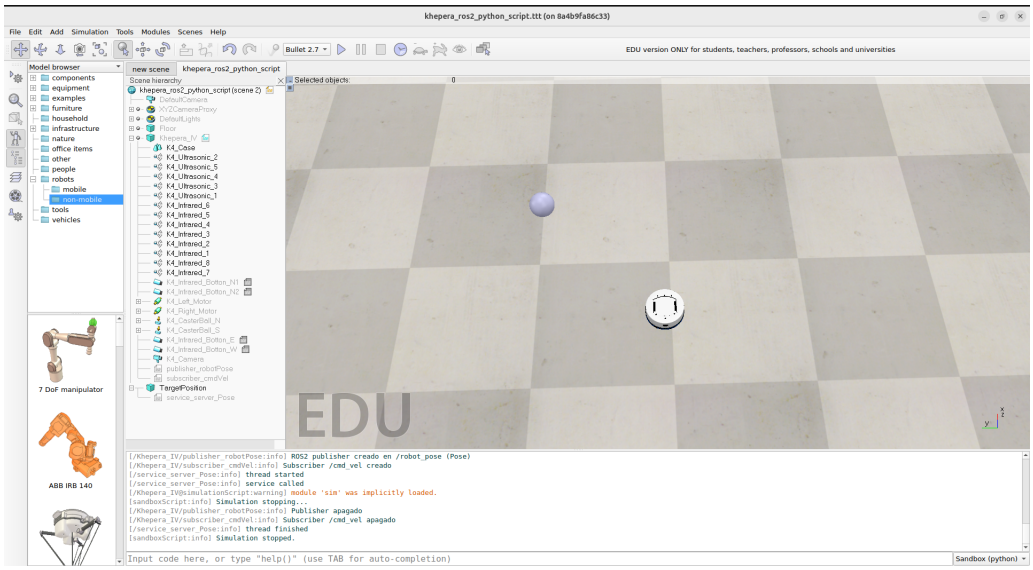

Figura 1: Robot *Khepera IV* y objeto que representa el punto de destino agregados a *Coppelia Sim*.

3. Iniciar la simulación y observar el comportamiento del robot. ¿A qué se debe este comportamiento?

4. Abrir el script de programación del robot y observar los valores de las velocidades izquierda y derecha.

El robot se mueve dibujando una trayectoria en arco de circunferencia. Haciendo doble click en el script por defecto del robot, Figura 30, se abre el script que se presenta en el Código 3. Este

comportamiento se debe a las velocidades angulares ($w_L,\ w_R$) asignadas a cada motor: 3 rad/s al izquierdo y 5 rad/s al derecho, respectivamente. Por tanto, el motor izquierdo gira más lento que el derecho y el robot realiza una trayectoria en arco de circunferencia de radio $R = 0{,}176$ metros, que se obtiene mediante la Ecuación (1) conociendo la distancia entre los centros de las ruedas (*axle track*), $L = 0{,}088$ metros [**kheperaiv_manual_rev3**].

$$R = \frac{L}{2} \cdot \frac{\omega_R + \omega_L}{\omega_R - \omega_L} \tag{1}$$

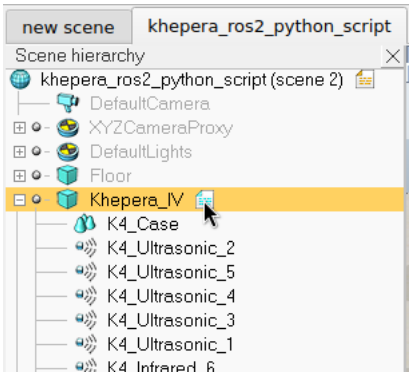

Figura 2: Script por defecto del robot.

5. Agregar a la simulación un objeto que represente el punto de destino del robot.

Se ha añadido una esfera con las propiedades descritas en la Figura 31 y en la posición $(x,\ y,\ z) = (0{,}75,\ 1{,}0,\ 0{,}05)$, como se observa en la Figura 29.

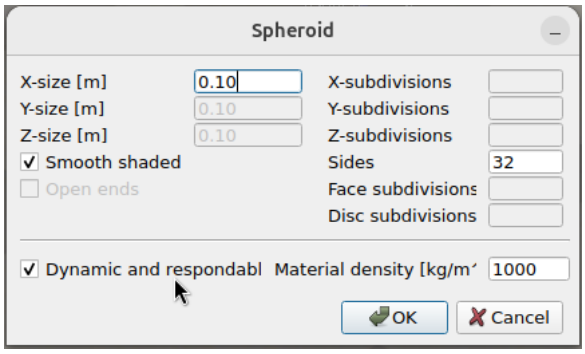

Figura 3: Propiedades de la esfera añadida como punto de destino.

6. Con las coordenadas del punto de destino y las del robot, calcular las velocidades angular y lineal para controlar su posición.

7. Obtener las velocidades que se envían al robot (izquierda y derecha) como resultado del modelo diferencial. Sugerencia: se puede comenzar solamente controlando la velocidad angular del robot a través el ángulo orientación hacia el punto y manteniendo la velocidad lineal constante, por medio de un control proporcional (P) sencillo. (*v*=cte, *w*=K_p*(error ángulo)).

8. Implementar una ley de control diferente a la proporcionada y comparar ambas usando los Índices de Comportamiento (*IAE*, *ISE*, *ITAE*, *ITSE*).

Dado que la implementación propuesta se ha realizado mediante *ROS2*, el primer paso es diseñar y definir el sistema de comunicación en *CoppeliaSim*. Para ello, se definen los siguiente requisitos:

- Se necesita obtener la posición del punto objetivo. Este destino es fijo, por lo que con obtener su posición al comienzo del programa es suficiente.
- Se necesita conocer la posición del robot en cada momento, ya que irá variando según las velocidades aplicadas.
- Se necesita accionar el robot con diferentes velocidades en cada momento, según el error calculado y la distancia restante.

En *ROS2* se dispone de tres vías de comunicación distintas: tópicos, servicios y acciones.

Un tópico es un canal de comunicación que permite el intercambio de mensajes de manera asíncrona, siguiendo el modelo publicador-suscriptor. Cada tópico está asociado a un tipo de mensaje específico. Los tópicos se utilizan, por ejemplo, para recibir información en tiempo real de los diferentes sensores del robot.

Un servicio es un mecanismo de comunicación síncrona basado en el modelo cliente-servidor que permite que un cliente envíe una petición a un servidor. El servidor procesa la solicitud y devuelve una respuesta al cliente. Son útiles para operaciones puntuales que requieren una respuesta inmediata, como pedir el estado de un sensor o sacar una foto. La comunicación mediante servicios involucra un mensaje de petición y otro de respuesta.

Una acción es una interfaz de comunicación asíncrona basada en tópicos y servicios diseñada para desempeñar operaciones largas y prolongadas en el tiempo que necesitan seguimiento. Para ejecutar una acción un cliente solicita a un servidor la ejecución, recibe retroalimentación periódica sobre el progreso a través de un tópico y tiene la opción de cancelar la tarea antes de que finalice. Son ideales para tareas como mover un robot a una posición determinada, donde es necesario monitorizar el avance y poder interrumpir la ejecución si es necesario. La comunicación de una acción involucra mensajes de objetivo, retroalimentación y resultado final.

Consecuentemente, el diseño de la arquitectura planteado, Figura 32, es el siguiente:

- Un tópico para publicar la posición del robot. El tópico se llama `/robot_pose` y utiliza el tipo de mensaje estándar `geometry_msgs/msg/Pose`. Este mensaje se compone de dos campos: un objeto *Point* (que se compone de tres números; x, y, z; en formato `float64`) y un objeto *Quaternion* (que se compone de cuatro números; x, y, z, w; en formato `float64`). Su

documentación está accesible en el siguiente enlace: `https://docs.ros.org/en/jazzy/p/geometry_msgs/__message_definitions.html`.

En *CoppeliaSim*, el robot tiene un script *Lua* llamado `publisher_robotPose` que ejerce de publicador de la posición, Código 4, mientras que el nodo o programa *ROS2* `khepera_controller`, externo a *CoppeliaSim*, Código 11, se suscribe al tópico.

- Un tópico para recibir los comandos de velocidad lineal y angular del robot. El tópico se llama `/cmd_vel` y utiliza el tipo de mensaje estándar `geometry_msgs/msg/Twist`. Este mensaje se compone de dos objetos *Vector3*: *linear* y *angular*. Ambos objetos se componen de tres campos cada uno; x, y, z; y representan la velocidad lineal o angular a aplicar en cada uno de los ejes. Su documentación también está accesible en el enlace anterior.

  En *CoppeliaSim*, el robot tiene un script *Lua* llamado `subscriber_cmdVel` que ejerce de suscriptor de la velocidad, Código 5, mientras que el nodo o programa *ROS2* `khepera_controller`, externo a *CoppeliaSim*, Código 11, publica en el tópico la velocidad deseada para comandar la velocidad al robot.

  Es importante destacar que el suscriptor implementado en *CoppeliaSim* transforma las velocidades lineales y angulares del robot comandadas en el tópico `/cmd_vel` en las velocidades individuales que debe ejecutar cada una de las ruedas del robot diferencial. La relación entre la velocidad lineal v y la velocidad angular ω del chasis con respecto a las velocidades angulares de las ruedas izquierda, ω_L, y derecha, ω_R viene dada por el modelo cinemático, Ecuaciones 2 en caso de utilizar velocidades angulares en cada rueda, o Ecuaciones 3 en caso de utilizar velocidades lineales, v_L y v_R, en cada rueda, donde r es el radio de las ruedas y L es la distancia entre ruedas. En este caso, se utilizan las Ecuaciones 2 porque el robot en uso controla los motores mediante velocidades angulares.

$$v = \frac{r}{2}(\omega_R + \omega_L) \qquad \omega = \frac{r}{L}(\omega_R - \omega_L) \tag{2}$$

$$v = \frac{v_L + v_R}{2} \qquad \omega = \frac{v_R - v_L}{L/2} \tag{3}$$

  Finalmente, despejando las velocidades angulares de cada rueda se obtiene la expresión necesaria para calcularlas, Ecuaciones 4.

$$\omega_L = \frac{v - \omega\frac{L}{2}}{r} \qquad \omega_R = \frac{v + \omega\frac{L}{2}}{r} \tag{4}$$

- Un servicio para solicitar la posición del objeto que representa el punto de destino en la trayectoria. El servicio se llama `/get_target_pose` y utiliza el tipo estándar `std_srvs/srv/Trigger`. Este tipo de servicio no recibe parámetros de entrada y devuelve una respuesta compuesta por dos campos: `success` (booleano) y `message` (cadena de texto). Su documentación está accesible en el siguiente enlace: `https://docs.ros.org/en/jazzy/p/std_srvs/index.html`.

  En *CoppeliaSim*, la esfera que representa el punto de destino dispone de un script *Lua* llamado `serviceServer_pose` que implementa el servidor del servicio, Código 6. Cuando el servicio es invocado desde el nodo *ROS2* externo `khepera_controller`, el servidor obtiene la posición y orientación del objeto. A continuación, empaqueta esta información en formato JSON dentro del campo `message` de la respuesta del servicio.

  Por su parte, el nodo ROS2 `khepera_controller`, Código 11, realiza la llamada al servicio `/get_target_pose` cada vez que necesita conocer la pose del punto de destino para actualizar la ley de control o generar la trayectoria correspondiente.

- Una acción para solicitar la ejecución de una trayectoria hacia un punto de destino. La acción se llama `/execute_trajectory` y utiliza el tipo personalizado `interfaces/action/ExecuteTrajectory`, Código 7. Esta acción está compuesta por tres estructuras: *Goal*, *Feedback* y *Result*. Su documentación está accesible en el siguiente enlace: `https://design.ros2.org/articles/actions.html`.

  La estructura *Goal* de esta acción contiene un campo de tipo `geometry_msgs/msg/Pose`. Si se le indica una posición, el robot inicia la trayectoria a esa posición. Si no se le indica, obtiene la posición de destino mediante el servicio `/get_target_pose`, solicitando de forma asíncrona la posición del punto de destino al servidor del servicio, evitando bloqueos en el *callback* de la acción. Como el movimiento se desarrolla en los ejes X e Y, no se tiene en cuenta el valor de Z (poner 0.0 por defecto).

  La estructura *Feedback* contiene información relevante sobre la evolución de la trayectoria en tiempo real. En concreto, incluye:

  - `distance`: distancia actual del robot al objetivo.
  - `e_theta`: error angular entre la orientación del robot y la dirección hacia el objetivo.
  - `v`: velocidad lineal que el controlador está aplicando.
  - `w`: velocidad angular aplicada.
  - `time_elapsed`: tiempo transcurrido desde el inicio de la acción.

  Esta información se publica periódicamente mientras el robot avanza hacia el objetivo, permitiendo monitorizar la evolución del control en tiempo real.

  La estructura *Result* contiene dos campos:

  - `success`: booleano que indica si el robot ha alcanzado el objetivo.
  - `message`: cadena de texto con información adicional sobre el resultado de la acción.

  El nodo *ROS2* externo `khepera_controller`, Código 11, implementa el servidor de la acción. Cuando un cliente envía una petición a `/execute_trajectory`, el servidor:

  1. Acepta la meta.
  2. Solicita de forma asíncrona la posición del objetivo mediante el servicio `/get_target_pose`.
  3. Una vez recibida la posición, inicia el bucle de control periódico.
  4. Publica *feedback* en cada iteración.
  5. Finaliza la acción cuando el robot alcanza el objetivo o si se produce una cancelación.

  Para realizar la solicitud de la acción desde terminal, ejecutar el Código 8. En dicho código aparece tanto la llamada indicando una posición como sin indicarla, en la que es suficiente con establecer el valor de cualquiera de los campos a "nan" para que obtenga mediante el servicio la posición del objeto como punto de destino.

- Un nodo *ROS2* encargado de implementar la lógica de control del robot y coordinar los distintos mecanismos descritos anteriormente. Este nodo, llamado `khepera_controller`, Código 11, actúa como elemento central del sistema, ya que:

  - se suscribe al tópico `/robot_pose` para recibir la pose del robot,
  - publica comandos de velocidad en el tópico `/cmd_vel`,

- invoca el servicio `/get_target_pose` para obtener la posición del objetivo,
- y expone la acción `/execute_trajectory` para permitir que un cliente solicite la ejecución de una trayectoria hacia dicho objetivo.

El nodo implementa un servidor de la acción `ExecuteTrajectory`. Cuando un cliente envía una petición, el nodo acepta la meta y solicita de forma asíncrona la posición del objetivo mediante el servicio `/get_target_pose`. Este diseño evita bloquear el *callback* de la acción y permite que el nodo continúe procesando otros eventos mientras espera la respuesta del servicio.

Una vez recibida la posición del objetivo, el nodo activa un bucle de control hasta alcanzar el objetivo. En cada iteración del bucle:

1. Se obtiene la pose actual del robot a partir del último mensaje recibido en el tópico `/robot_pose`.
2. Se calcula el error de posición y orientación respecto al objetivo.
3. Se aplica la ley de control seleccionada, generando las velocidades lineal v y angular ω.
4. Se publican dichas velocidades en el tópico `/cmd_vel`.
5. Se envía al cliente de la acción un mensaje de *feedback* con información sobre la evolución de la trayectoria: distancia al objetivo, error angular, velocidades aplicadas y tiempo transcurrido.

El nodo considera que el objetivo ha sido alcanzado cuando la distancia al punto de destino es inferior a un umbral predefinido (por ejemplo, 5 centímetros).

Además, este nodo incorpora un módulo específico para la evaluación cuantitativa del rendimiento del controlador. Este módulo, implementado en la clase `MetricsLogger`, Códigos 15 y 16, registra en cada iteración del bucle de control tanto las variables relevantes del estado del robot (posición, orientación, distancia al objetivo y errores de control) como las velocidades lineal y angular aplicadas. A partir de estos datos, el sistema calcula varias métricas clásicas de evaluación de control, tales como el *Integral of Absolute Error* (*IAE*), el *Integral of Squared Error* (*ISE*), el *Integral of Time-weighted Absolute Error* (*ITAE*) y el *Integral of Time-weighted Squared Error* (*ITSE*). Estas métricas permiten analizar la calidad de la trayectoria ejecutada y comparar el comportamiento de distintos parámetros o estrategias de control.

Al finalizar la acción, el nodo almacena toda esa información en un fichero *CSV*, incluyendo una fila por cada instante de tiempo con las variables medidas y calculadas, seguida de un bloque final con los valores agregados de las métricas y los parámetros del controlador utilizados (`kp`, `vmax`, `wmax` y el tipo de controlador seleccionado).

Por último, mencionar que para facilitar la depuración y modularidad del código, en lugar de implementar todas las responsabilidades en el propio `khepera_controller`, se han definido tanto las cabeceras como las funciones en distintos módulos.

El módulo `service_utils.hpp/.cpp` contiene las funciones auxiliares para realizar la llamada asíncrona al servicio `/get_target_pose`, Códigos 12 y 13, el módulo `pose_utils.hpp` proporciona utilidades para extraer y manipular la pose del robot (posición y orientación) a partir de los mensajes recibidos, Código 14, y el módulo `metrics_utils.hpp/.cpp` implementa el sistema de evaluación y registro de métricas, Códigos 15 y 16.

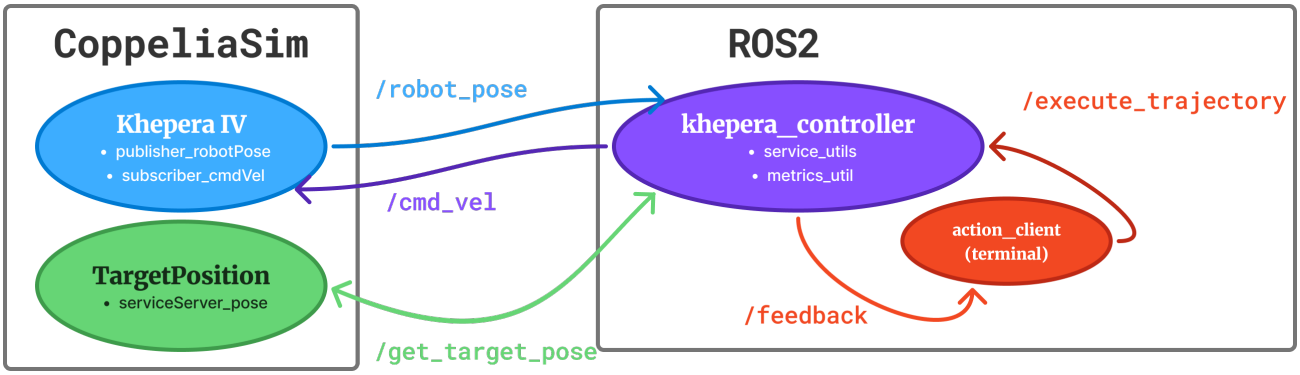

Figura 4: Esquema de la arquitectura propuesta.

Controladores implementados

En esta práctica se implementan y comparan dos controladores cinemáticos para un robot diferencial basados en el concepto de *Instantaneous Center of Curvature* (ICC). El primero es un controlador proporcional reactivo (control-P) hacia un objetivo, mientras que el segundo es un controlador geométrico basado en curvatura, equivalente al enfoque *Pure Pursuit* puntual. Ambos enfoques están ampliamente descritos en la literatura clásica de control de robots móviles [**choset2005**, **siegwart2011**, **siciliano2009**].

Geometría del problema

Sea la posición del robot (x_r, y_r) y la del objetivo (x_t, y_t), se define:

$$d = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2} \tag{5}$$

$$\theta_d = \text{atan2}(y_t - y_r,\ x_t - x_r) \tag{6}$$

$$e_\theta = \theta_d - \theta_r \tag{7}$$

donde d es la distancia al objetivo, e_θ el error de orientación del robot respecto del objetivo, θ_d el ángulo absoluto definido en el marco global de la orientación de la línea recta que une la posición del robot con la del objetivo y θ_r la orientación actual del robot. De este modo, θ_d indica la dirección hacia la que debería orientarse el robot para apuntar directamente al objetivo, mientras que el error angular $e_\theta = \theta_d - \theta_r$ mide la diferencia entre dicha orientación deseada y la orientación actual del robot. Esta formulación es estándar en control cinemático de robots diferenciales [**otero2005**, **morin2016**].

En la Figura 33 se presenta el esquema de un robot móvil diferencial con ruedas.

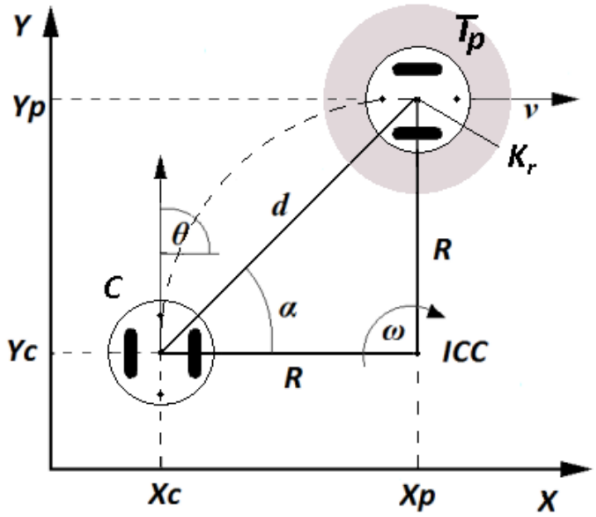

Figura 5: Esquema de un robot móvil diferencial con ruedas [**FARIAS20179150**].

1. Controlador proporcional reactivo

El controlador reactivo utiliza una ley proporcional para la velocidad lineal,

$$v = K_p \cdot d \ , \tag{8}$$

saturada por la velocidad máxima del robot:

$$v = \text{mín}(v,\ v_{\text{máx}}) \ . \tag{9}$$

La velocidad angular se calcula como:

$$\omega = \omega_{\text{máx}} \cdot \sin(e_\theta) \tag{10}$$

Este tipo de controlador proporcional basado en errores de pose es ampliamente utilizado en robots uniciclo y se describe en [**choset2005**, **siegwart2011**].

2. Controlador geométrico basado en curvatura o *Pure Pursuit* puntual

El controlador geométrico impone que el robot siga un arco de circunferencia que conecta su pose actual con el objetivo. El radio del arco viene dado por:

$$R = \frac{d}{2\sin(e_\theta)} \tag{11}$$

La velocidad angular se obtiene mediante la relación cinemática:

$$\omega = \frac{v}{R} \tag{12}$$

Este enfoque corresponde a la formulación puntual del algoritmo *Pure Pursuit*, descrito originalmente en [**coulter1992**] y ampliamente analizado en [**otero2005**, **morin2016**, **corke2017**]. A diferencia del controlador reactivo, la forma de la trayectoria depende únicamente de la geometría del problema.

Evaluación

Para comparar ambos controladores se han ejecutado trayectorias desde la posición inicial del robot, ($x = 0{,}27887,\ y = 0{,}525$), hasta la posición de destino, ($x = 0{,}75,\ y = 1{,}0$), estableciendo la velocidad máxima del robot a $1m/s$, [**kheperaiv_manual_rev3**] y realizando un barrido del valor de K_p, comenzando en $0{,}1$ y aumentando en $0{,}05$ hasta $0{,}6$, ya que con una ganancia superior la aceleración del robot es excesiva y provoca el volcado del mismo. El control de la aceleración queda fuera del alcance de esta tarea.

La evaluación de los controladores se basa en el cálculo y comparación de las siguientes métricas estándar en control:

- **Integral del Error Absoluto (*IAE*)**: mide el error acumulado a lo largo del tiempo, proporcionando una valoración global del desempeño sin penalizar en exceso los errores grandes. Es adecuada cuando se desea evaluar el comportamiento medio del sistema o cuando los errores puntuales no son críticos.

$$IAE = \int_0^T |e(t)|\, dt \tag{13}$$

- **Integral del Error Cuadrático (*ISE*)**: penaliza de forma más severa los errores grandes, siendo útil para evaluar controladores que deben minimizar desviaciones significativas o evitar comportamientos agresivos. Es apropiada cuando los errores grandes son especialmente indeseables.

$$ISE = \int_0^T e(t)^2\, dt \tag{14}$$

- **Integral del Error Absoluto Ponderado por el Tiempo (*ITAE*)**: penaliza especialmente los errores que persisten en el tiempo, favoreciendo respuestas rápidas y con menor cola temporal. Se utiliza cuando el objetivo es minimizar el tiempo de asentamiento y mejorar la rapidez de convergencia.

$$ITAE = \int_0^T t\,|e(t)|\, dt \tag{15}$$

- **Integral del Error Cuadrático Ponderado por el Tiempo (*ITSE*)**: combina la penalización de errores grandes con la penalización de errores tardíos, siendo útil para evaluar estabilidad y convergencia en sistemas no lineales. Es adecuada cuando interesa evitar tanto errores grandes como oscilaciones prolongadas.

$$ITSE = \int_0^T t\, e(t)^2\, dt \tag{16}$$

En la práctica, estas métricas deben implementarse de manera discreta, por lo que dado el periodo de muestreo Δt, las métricas se aproximan mediante sumas:

$$IAE \approx \sum_{k=0}^{N} |e_k|\, \Delta t \tag{17}$$

$$ISE \approx \sum_{k=0}^{N} e_k^2\, \Delta t \tag{18}$$

$$\textit{ITAE} \approx \sum_{k=0}^{N} t_k \, |e_k| \, \Delta t \tag{19}$$

$$\textit{ITSE} \approx \sum_{k=0}^{N} t_k \, e_k^2 \, \Delta t \tag{20}$$

Estas métricas permiten comparar objetivamente el rendimiento de ambos controladores bajo diferentes valores de K_p.

Registro de datos

En cada iteración del controlador se almacena el tiempo t, la pose del robot (x_r, y_r, θ_r), la pose del objetivo (x_t, y_t), error de distancia d, error de orientación e_θ y velocidades v y ω.

Al finalizar la acción, el sistema añade al fichero *CSV* las métricas globales calculadas (*IAE*, *ISE*, *ITAE*, *ITSE*) y los parámetros utilizados (K_p, $v_{\text{máx}}$, $\omega_{\text{máx}}$ y tipo de controlador). Este registro permite realizar comparativas rigurosas mediante tablas y gráficas.

Resultados

En las Figuras 34 a 44 se presentan las gráficas que dibujan las distintas señales medidas durante la ejecución de la trayectoria del primer controlador.

En las Figuras 45 a 55 se presentan las gráficas que dibujan las distintas señales medidas durante la ejecución de la trayectoria del segundo controlador.

En la Tabla 3 se muestran las cuatro métricas de evaluación calculadas para cada controlador y cada configuración, donde se ha coloreado en amarillo los valores más bajos obtenidos para cada métrica.

Estos resultados evidencian que el controlador proporcional reactivo mantiene un desempeño consistente a lo largo de distintos valores de K_p, con reducciones progresivas en todas las métricas al aumentar la ganancia, por lo que disminuyen tanto los errores acumulados como los errores tardíos y los picos de error, reflejando un comportamiento estable y uniforme.

El controlador geométrico basado en curvatura, por su parte, presenta inicialmente valores de error mayores, especialmente en *IAE* e *ITAE*, lo que refleja errores acumulados y tardíos superiores para valores bajos de K_p. Sin embargo, a medida que la ganancia aumenta, la reducción de todas las métricas es más pronunciada que en el controlador proporcional. Esta mayor pendiente de mejora indica que, aunque al principio su desempeño es inferior, el controlador geométrico alcanza rápidamente un rendimiento superior. De hecho, para $K_p = 0{,}5$, logra los menores valores en todas las métricas, lo que demuestra que minimiza eficazmente todos los errores, superando al controlador proporcional reactivo en precisión y rapidez de convergencia.

Conclusiones

La comparación evidencia que el controlador proporcional reactivo es más estable y uniforme a lo largo de los valores de la ganancia K_p, mientras que el controlador geométrico tiene un

desempeño más sensible a la ganancia, pero capaz de superar al reactivo cuando la ganancia alcanza valores óptimos. Esta situación resalta la importancia de ajustar correctamente las ganancias para aprovechar la mayor capacidad de seguimiento del controlador geométrico, especialmente cuando se busca minimizar errores tardíos y grandes y maximizar la precisión global en la trayectoria.

		0.1	**0.15**	**0.2**	**0.25**	**0.3**	**0.35**	**0.4**	**0.45**	**0.5**	**0.55**	**0.6**
Cont. prop. reactivo	***IAE***	2.83	1.91	1.46	1.18	1.03	0.92	0.81	0.74	0.76	0.86	0.85
	ISE	1.05	0.73	0.56	0.46	0.42	0.38	0.33	0.31	0.33	0.39	0.43
	ITAE	9.99	4.47	2.58	1.66	1.21	0.97	0.74	0.61	0.63	0.75	0.66
	ITSE	2.29	1.06	0.61	0.41	0.31	0.25	0.19	0.17	0.18	0.24	0.24
Cont. geom.	***IAE***	3.16	2.16	1.64	1.28	1.05	0.95	0.76	0.77	0.7	0.75	0.88
	ISE	1.21	0.83	0.64	0.51	0.42	0.38	0.31	0.32	0.3	0.34	0.45
	ITAE	11.94	5.57	3.17	1.92	1.29	1.03	0.67	0.67	0.55	0.59	0.7
	ITSE	2.9	1.36	0.79	0.48	0.33	0.27	0.17	0.18	0.15	0.18	0.26

Tabla 1: Evaluación de los controladores con las cuatro métricas propuestas para distintos valores de K_p.

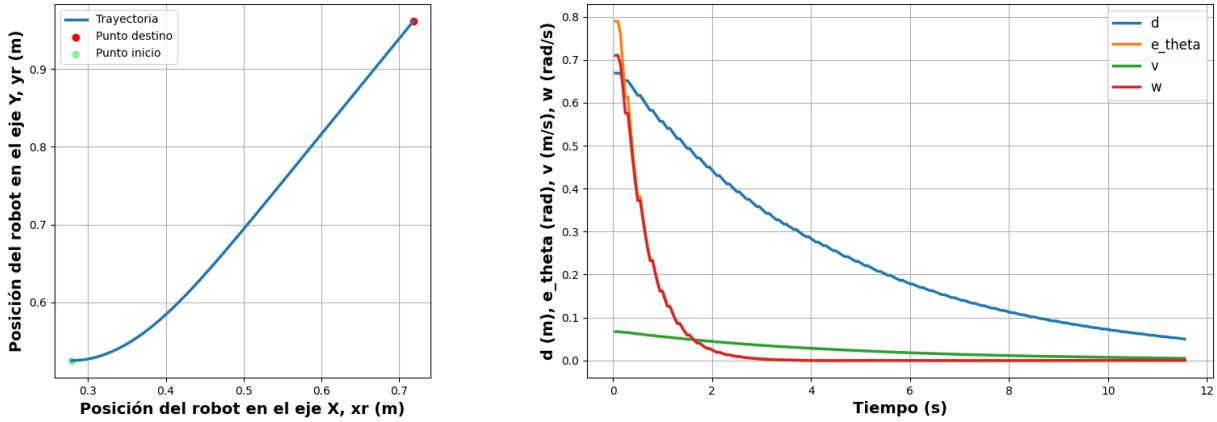

Figura 6: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}1$.

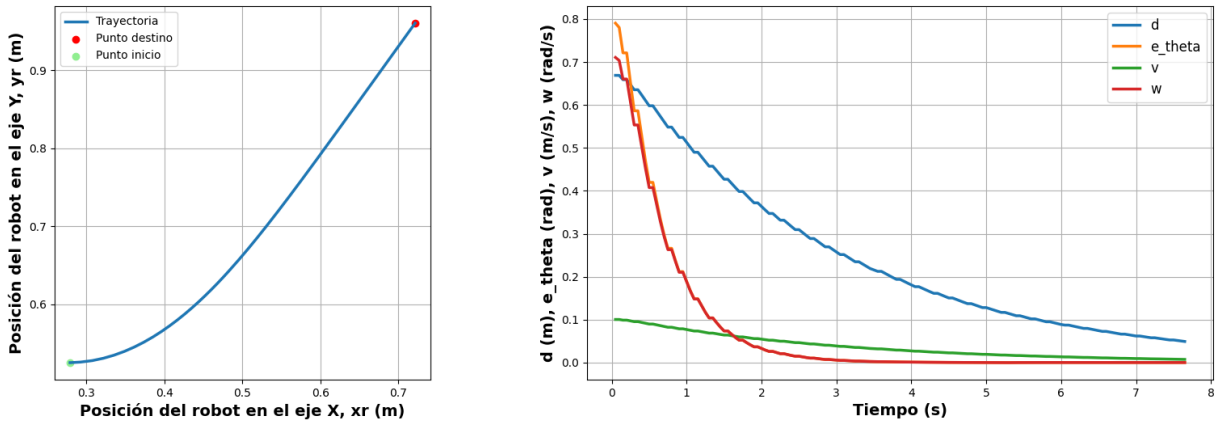

Figura 7: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}15$.

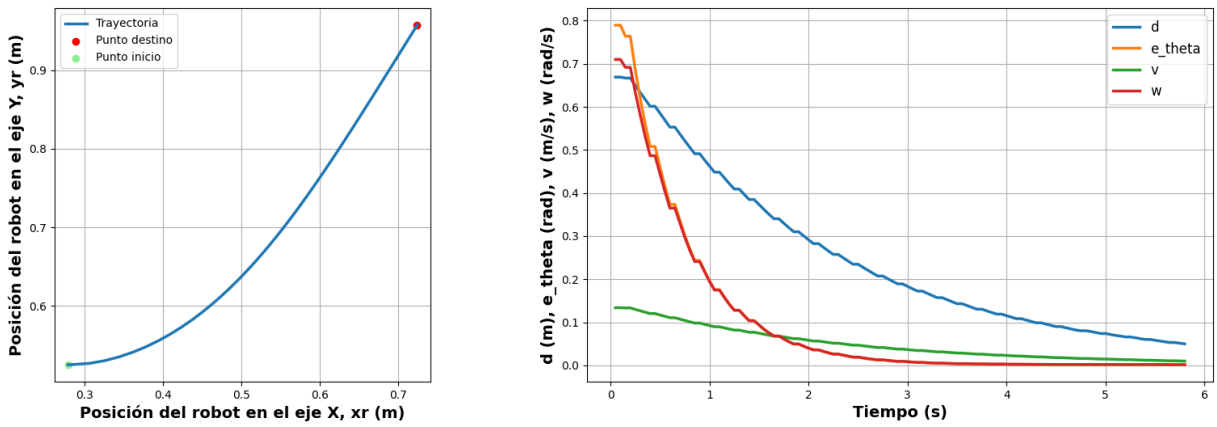

Figura 8: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}2$.

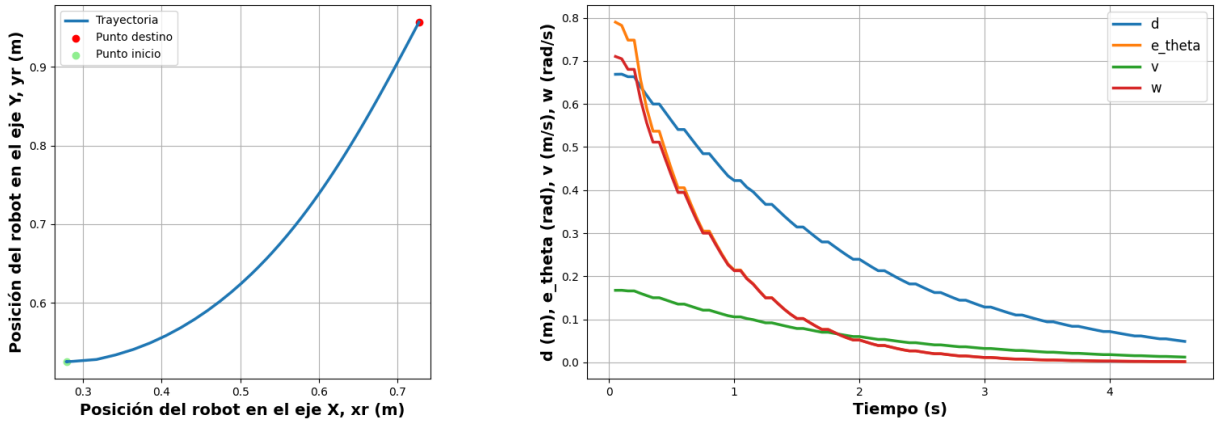

Figura 9: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}25$.

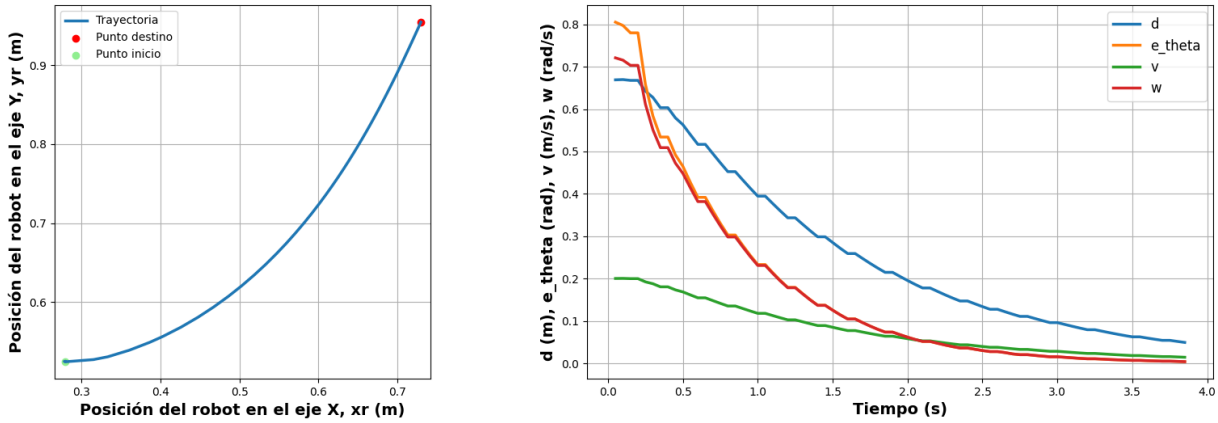

Figura 10: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}3$.

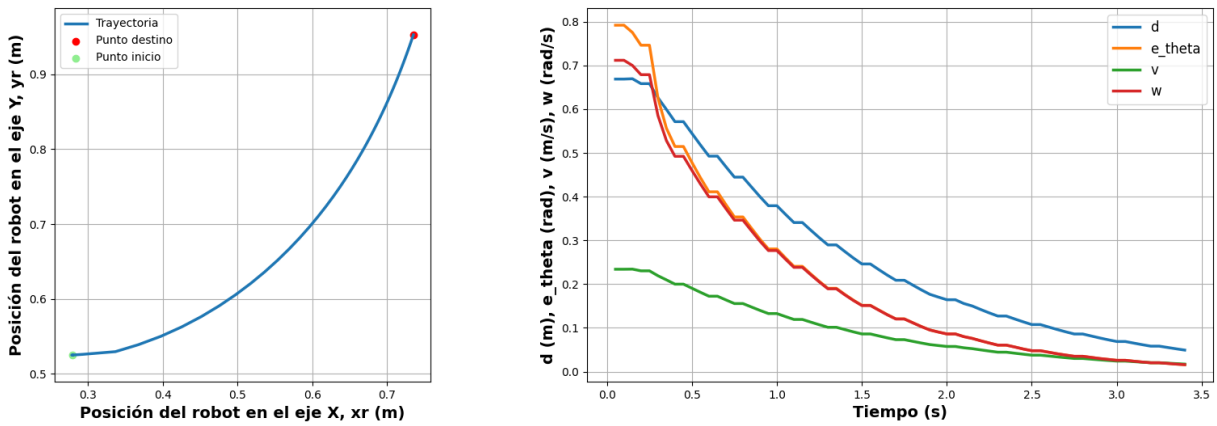

Figura 11: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}35$.

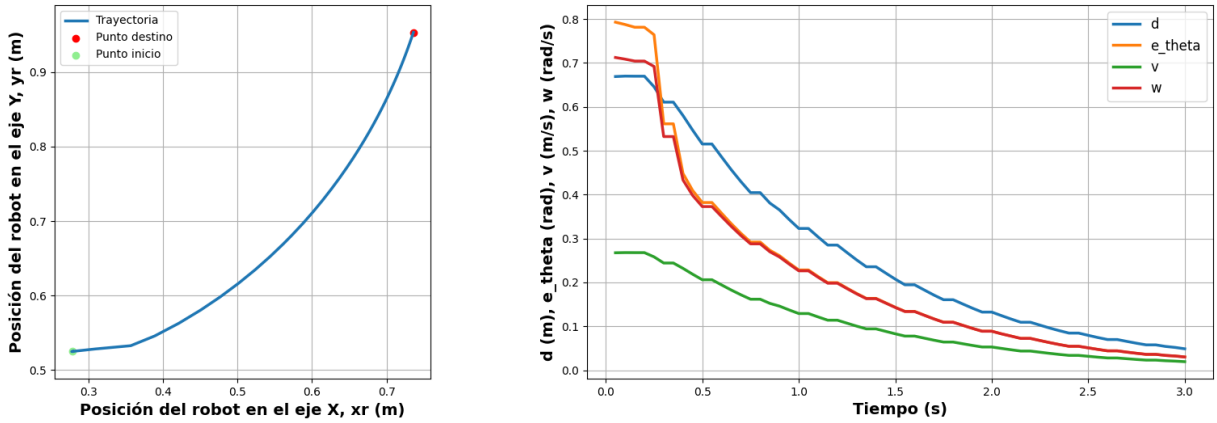

Figura 12: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}4$.

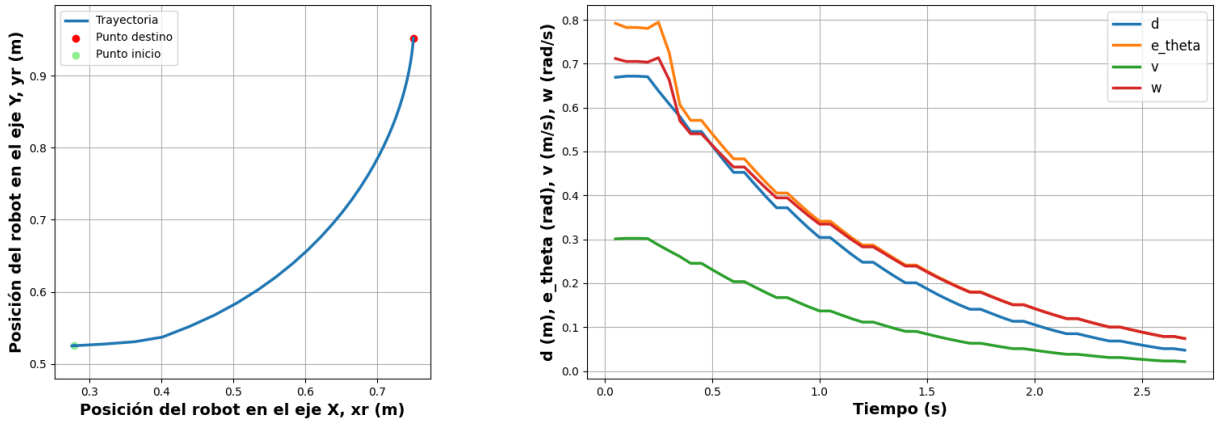

Figura 13: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}45$.

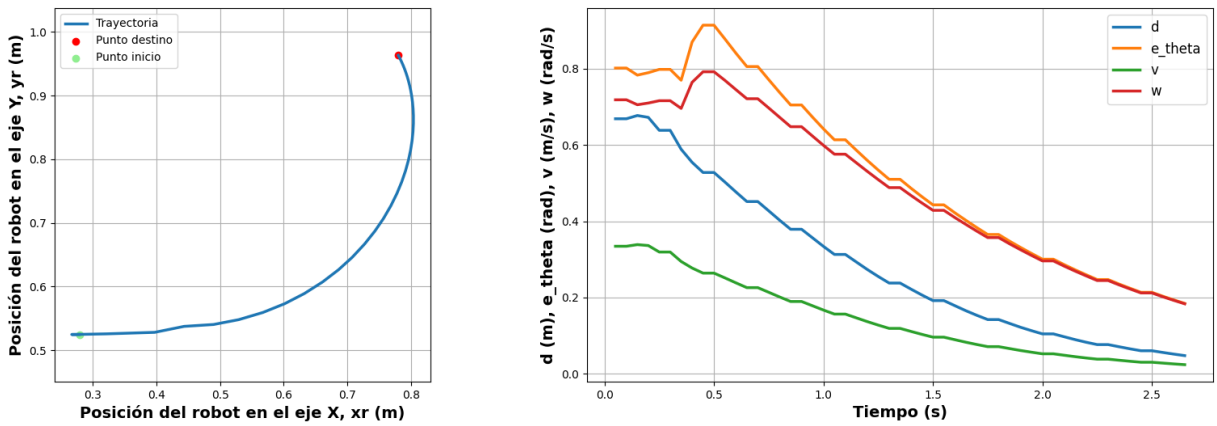

Figura 14: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}5$.

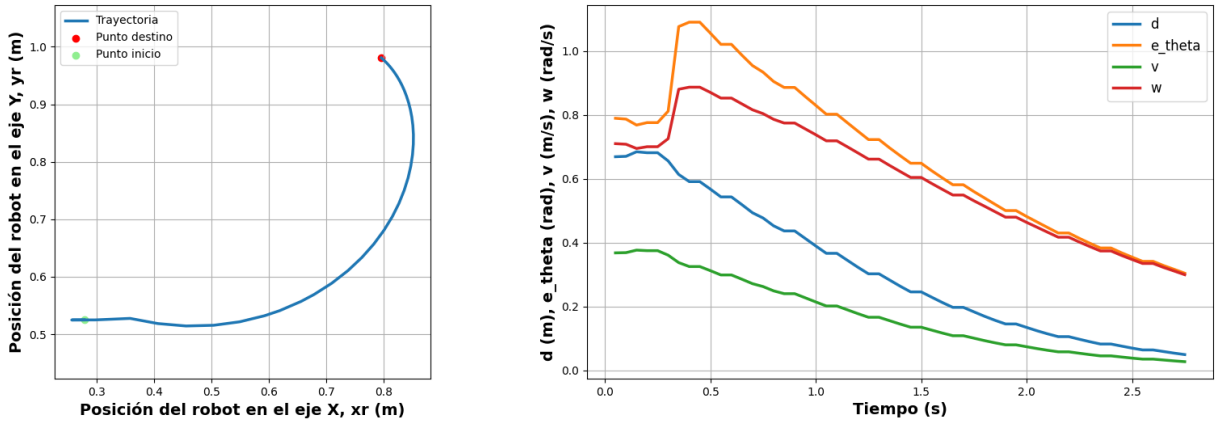

Figura 15: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}55$.

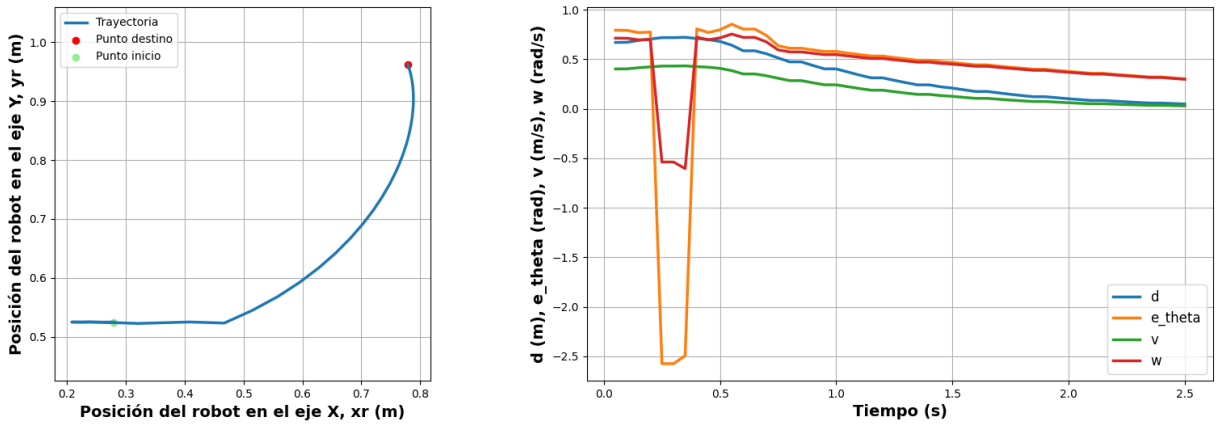

Figura 16: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}6$.

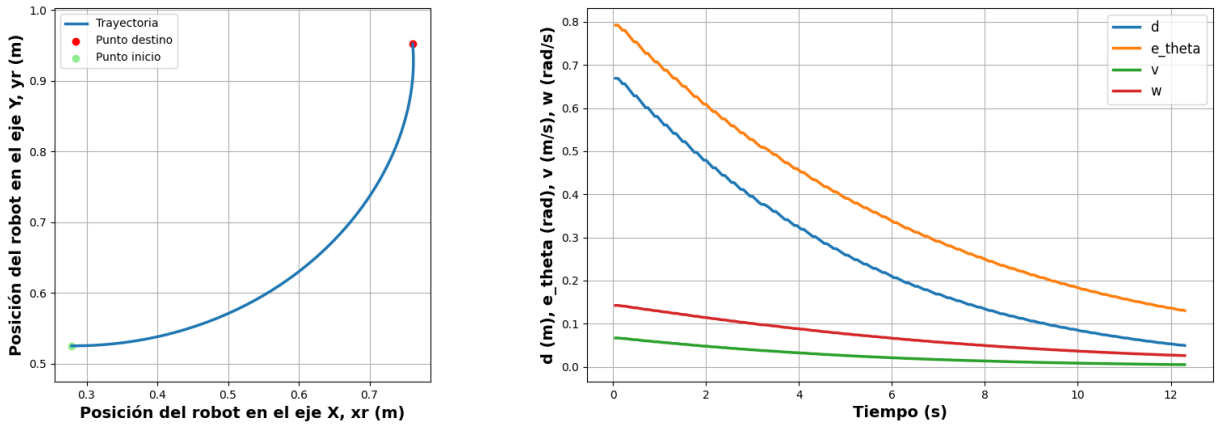

Figura 17: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}1$.

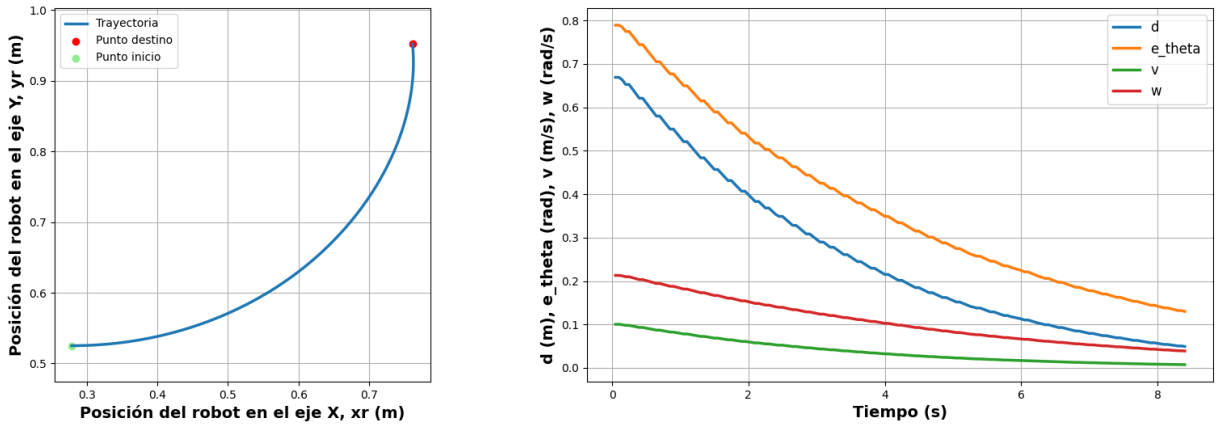

Figura 18: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}15$.

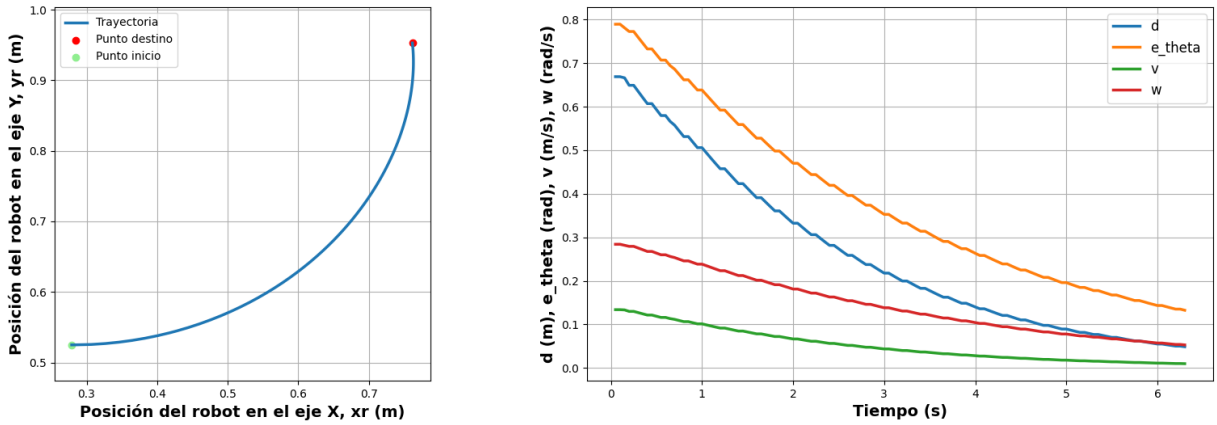

Figura 19: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}2$.

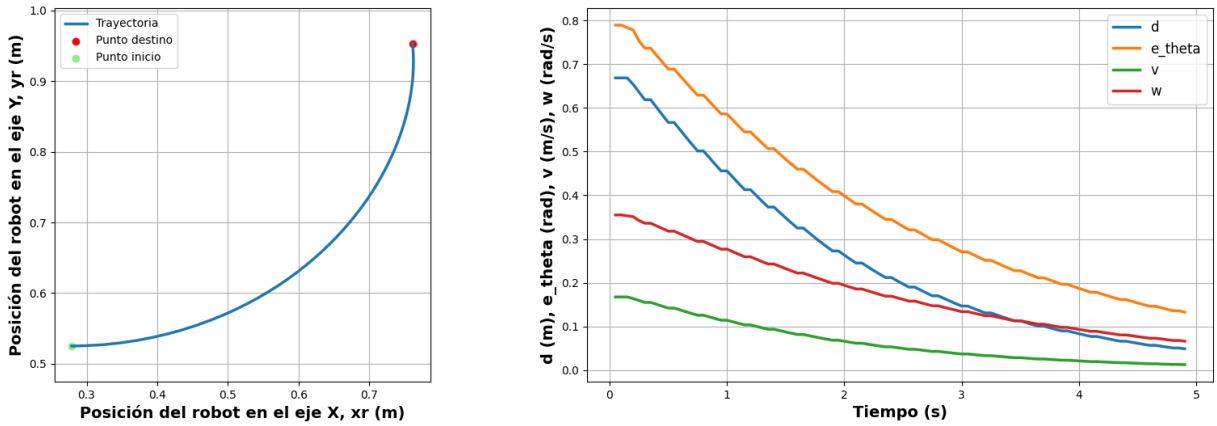

Figura 20: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}25$.

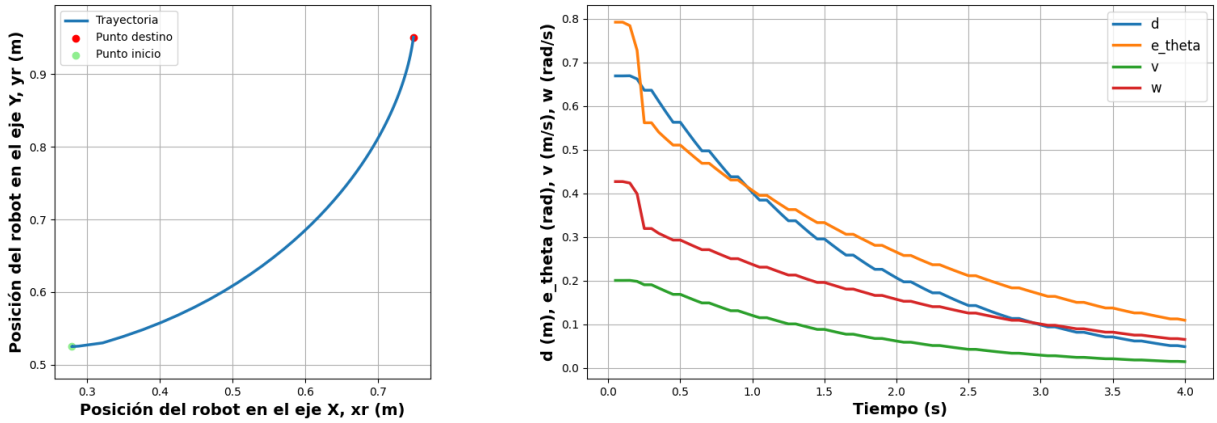

Figura 21: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}3$.

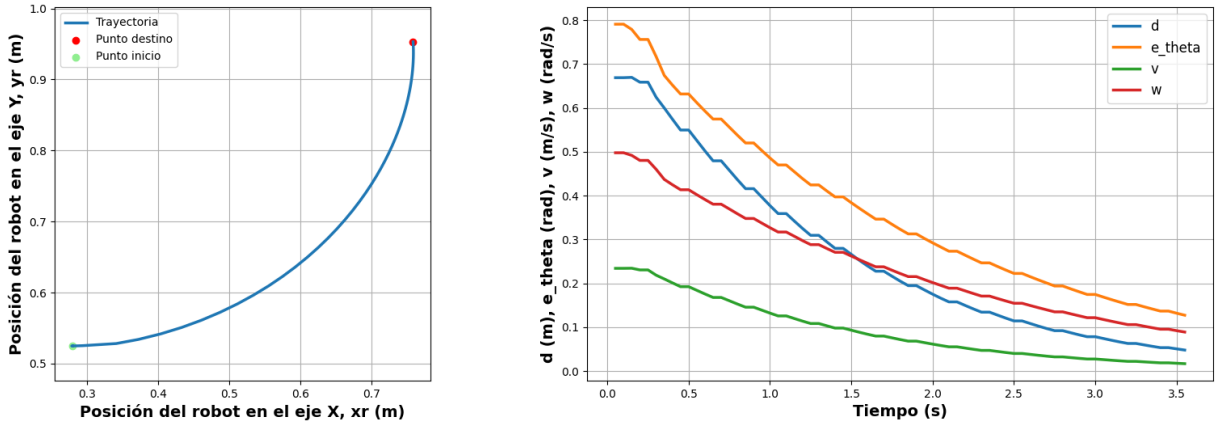

Figura 22: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}35$.

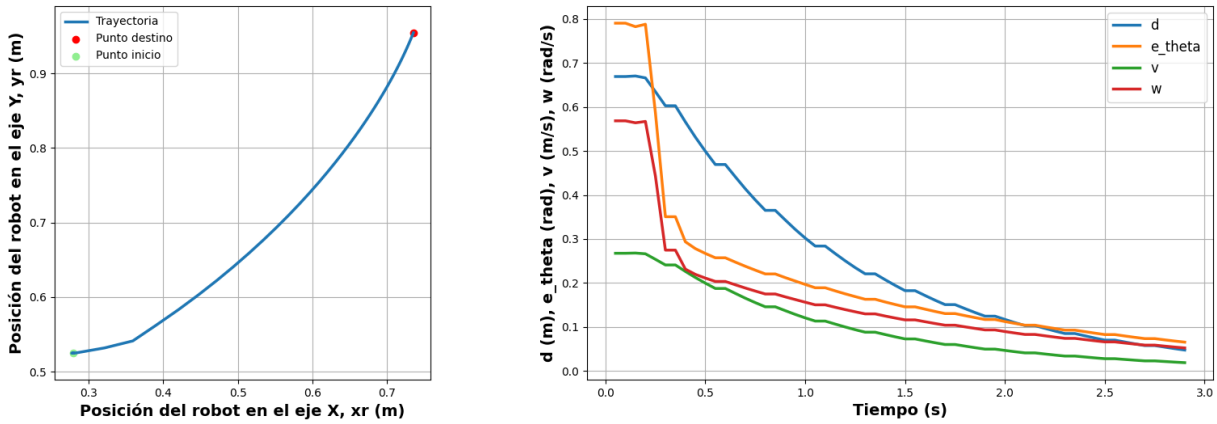

Figura 23: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}4$.

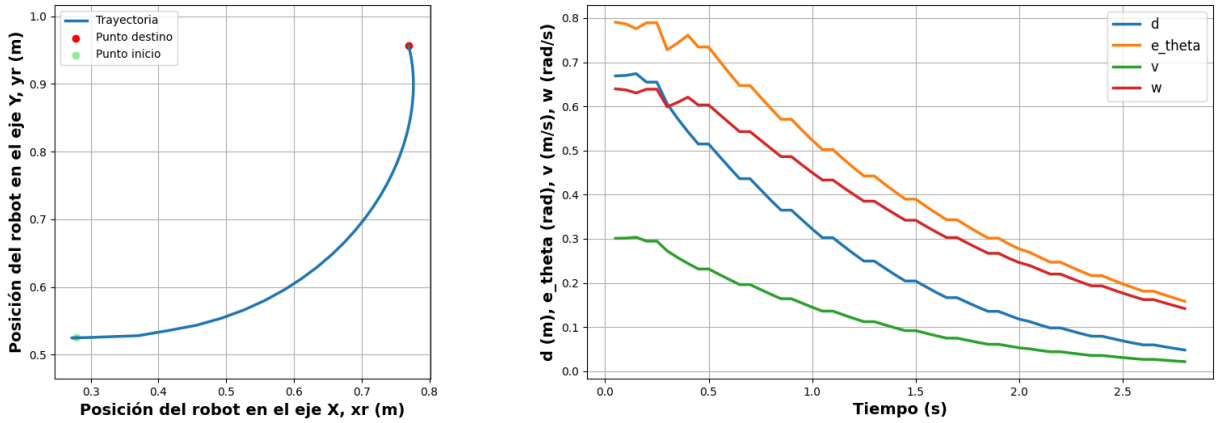

Figura 24: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}45$.

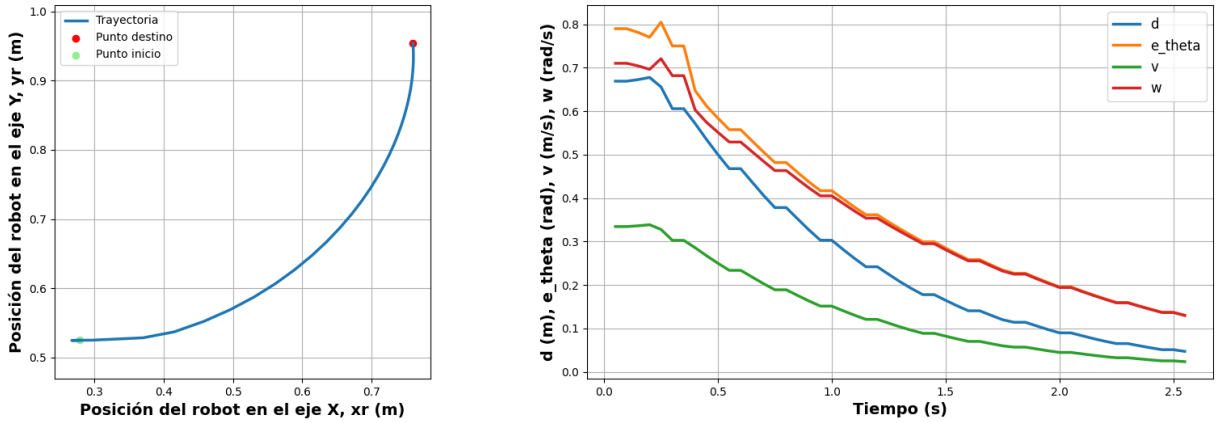

Figura 25: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}5$.

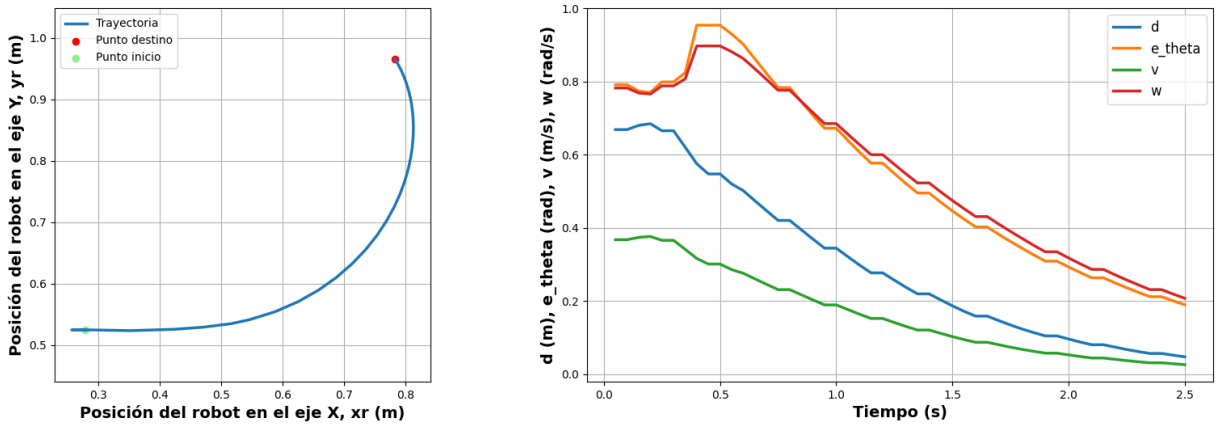

Figura 26: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}55$.

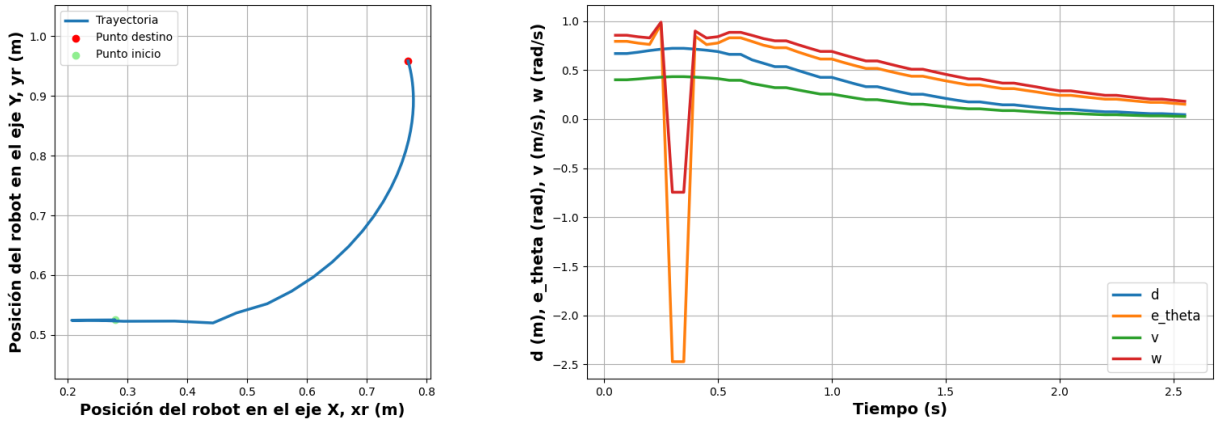

Figura 27: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}6$.

Tarea 2: control de posición con evitación de obstáculos.

El experimento consiste en controlar la posición del robot adicionando obstáculos al experimento:

1. Agregar obstáculos al espacio de trabajo, pueden ser paredes, cilindros, etc... nota: habilitar la propiedad "detectable" de los obstáculos para que el robot los pueda detectar con los sensores.

IMAGEN XX para ver dónde poner detectable y imágenes escenarios.

2. Implementar el algoritmo de *Braitenberg* por medio de la manipulación de las velocidades del robot.

El algoritmo de *Braitenberg* se refiere a aquellos procesos de control de vehículos reactivos (o de *Braitenberg*) que relacionan directamente señales sensoriales a actuadores mediante ganancias simples, generando comportamientos complejos a partir de reglas locales [**braitenberg1984vehicles**]. En otras palabras, el concepto de vehículos de *Braitenberg* corresponde a aquellos cuyos actuadores están directamente gobernados por los sensores, esto es, los sensores están conectados a los actuadores.

Este esquema describe comportamientos reactivos y en la práctica, esto se implementa mapeando lecturas de sensores s_i a comandos de ruedas mediante matrices de ganancia, sin planificación ni memoria interna.

En la Figura 28 se ilustran estos vehículos, donde las flechas de color negro representan la conexión directa `sensor` $\rightarrow$ `actuador`, mientras que la de color azul el movimiento reactivo del robot según la lectura de los sensores. El círculo de color rojo representa un obstáculo a esquivar.

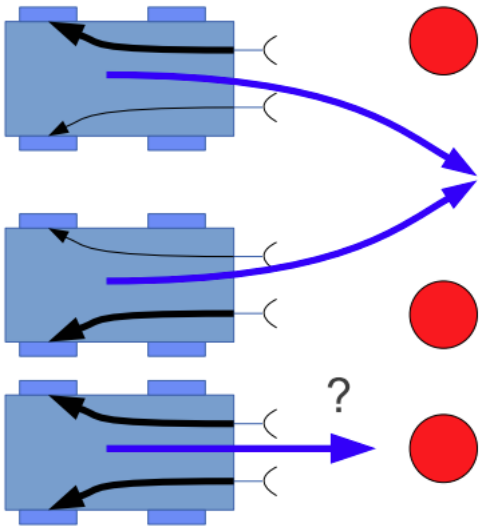

Figura 28: Vehículos de *Braitenberg* [**dicaro2017behaviors**].

Un esquema *Braitenberg* clásico para un robot diferencial se expresa como

$$v_L \;=\; \sum_i G_{L,i}\, f(s_i), \qquad v_R \;=\; \sum_i G_{R,i}\, f(s_i), \tag{21}$$

donde $G_{L,i}, G_{R,i}$ son las ganancias que conectan el sensor i con las ruedas, $f(\cdot)$ es el preprocesado (inversión, saturación, normalización, filtrado, etc.) y v_L, v_R son las velocidades lineales de las ruedas (m/s).

La velocidad lineal del chasis v y su velocidad angular ω se obtienen de las Ecuaciones 2 o 3. Como el robot controla los motores mediante velocidades angulares de rueda, las velocidades lineales de rueda se convierten a velocidades angulares mediante el radio de rueda r, Ecuaciones 22, obteniendo así las Ecuaciones 2.

$$\omega_L \;=\; \frac{v_L}{r}, \qquad \omega_R \;=\; \frac{v_R}{r}. \tag{22}$$

Métodos comunes de implementación

- **Conexión directa (clásica)**: cada sensor contribuye linealmente a las ruedas; muy simple y reactivo.
- **Cross-coupled / contralateral**: sensores frontales izquierdos conectan a la rueda derecha (y viceversa) para producir evasión frontal más efectiva.
- **Ipsilateral con inversión**: sensor izquierdo a rueda izquierda con signo invertido para comportamientos de atracción (seguir fuente).
- **Normalizado y ponderado**: normalizar activaciones y aplicar pesos dependientes de la posición del sensor para evitar que una lectura domine.
- **Mezcla con controlador global**: combinar la salida Braitenberg con un controlador de objetivo (campo potencial, PID, etc.) mediante un factor α:

$$v = \alpha\, v_{\text{goal}} + (1-\alpha)\, v_{\text{brait}}, \quad w = \alpha\, w_{\text{goal}} + (1-\alpha)\, w_{\text{brait}}.$$

- **Híbridos y arquitecturas de comportamiento**: usar Braitenberg como uno de varios comportamientos en una arquitectura de subsunción o en un arbitrador basado en utilidades.
- **Aprendido / adaptativo**: ajustar las ganancias G mediante aprendizaje (regresión, optimización) para adaptar el comportamiento a la dinámica real del robot.

Formalización

Preprocesado y buenas prácticas

- **Mapeo distancia $\rightarrow$ activación:** si d_i es la distancia medida por el sensor i,

$$a_i = \text{clamp}\left(\frac{d_{\text{máx}} - d_i}{d_{\text{máx}} - d_{\text{mín}}}, 0, 1\right),$$

  y para sensores IR conviene invertir (cercanía $\rightarrow$ alta activación).

- **Filtrado exponencial:** $a_i^{(t)} = \beta\, a_i^{(t-1)} + (1-\beta)\, a_{i,\text{raw}}$ para reducir ruido.
- **Normalización y saturación:** dividir por $\sum_i |G_{*i}|$ o aplicar saturación final a v_L, v_R.
- **Diferenciar sensores:** usar ganancias distintas para frontales y laterales; atenuar lateral en pasillos.
- **Seguridad:** límites de velocidad, limitación de aceleración y freno por distancia frontal.

Ejemplos de matrices de conexión

A continuación se muestran matrices de ejemplo para un robot con 8 sensores IR ordenados (RL, L, FL, F, FR, R, RR, Rear). Los valores son ilustrativos; deben ajustarse experimentalmente.

$$G_{\text{contralateral}} = \begin{bmatrix} G_{L,RL} & G_{L,L} & G_{L,FL} & G_{L,F} & G_{L,FR} & G_{L,R} & G_{L,RR} & G_{L,Rear} \\ G_{R,RL} & G_{R,L} & G_{R,FL} & G_{R,F} & G_{R,FR} & G_{R,R} & G_{R,RR} & G_{R,Rear} \end{bmatrix} = \begin{bmatrix} -0{,}2 & -0{,}4 & -0{,}6 & -0{,}8 \\ 0{,}2 & 0{,}4 & 0{,}6 & 0{,}8 \end{bmatrix}$$

$$G_{\text{ipsilateral_atraccion}} = \begin{bmatrix} 0{,}3 & 0{,}4 & 0{,}5 & 0{,}6 & 0{,}5 & 0{,}4 & 0{,}3 & 0{,}1 \\ 0{,}3 & 0{,}4 & 0{,}5 & 0{,}6 & 0{,}5 & 0{,}4 & 0{,}3 & 0{,}1 \end{bmatrix}$$

Pseudocódigo (implementación práctica por ciclo de control)

```
1. Leer sensores d[i]
2. a[i] = preprocess(d[i])    // normalizar, invertir, filtrar
3. vL = sum_i G_L[i] * a[i]
   vR = sum_i G_R[i] * a[i]
4. v_brait = (vL + vR)/2
   w_brait = k_diff * (vR - vL)
5. (v_field, w_field) = campo_vectorial(...)  // si se usa
6. v_des = alpha * v_field + (1-alpha) * v_brait
   w_des = alpha * w_field + (1-alpha) * w_brait
7. aplicar limitación de aceleración y saturaciones
8. publicar cmd_vel
```

Comparativa resumida

Método	Simplicidad	Robustez	Uso típico
Directo clásico	Alta	Baja	Robots educativos, prototipos
Cross-coupled	Alta	Media	Evasión frontal
Mezcla con campo	Media	Alta	Objetivo + evasión
Normalizado/ponderado	Media	Alta	Entornos ruidosos
Híbrido / subsunción	Baja	Muy alta	Sistemas reales
Aprendido	Baja	Variable	Ajuste automático

Tabla 2: Comparativa de métodos de implementación de Braitenberg.

Problemas típicos y mitigaciones

- **Bloqueo en pasillos (mínimos locales):** romper simetría con un *bias* lateral, modo corredor (atenuar repulsión lateral y añadir un pequeño impulso hacia adelante) o activar wall·following.
- **Choques por atracción dominante:** mezclar con controlador objetivo con factor α dependiente de la distancia frontal, aumentar k_{rep} o elevar $d_{\text{mín}}$.
- **Oscilaciones / tumble:** limitar aceleraciones, usar factor de alineamiento $\cos(e_\theta)$ para reducir avance con gran error angular y añadir término derivativo en la acción angular.

Variantes implementadas en este trabajo

1. **Variante A (Braitenberg puro contralateral):** matriz G contralateral para evasión frontal; activaciones a_i normalizadas y filtradas; salida directa a ruedas.
2. **Variante B (Braitenberg mezclado con campo vectorial):** salida Braitenberg como término de evasión $(v_{\text{brait}}, w_{\text{brait}})$ y campo vectorial como término de objetivo $(v_{\text{field}}, w_{\text{field}})$; mezcla ponderada por $\alpha(d_{\text{front}})$ que disminuye la influencia del campo cuando la distancia frontal es pequeña.
3. **Variante C (fallback wall·following):** detección de estancamiento en pasillo y conmutación a controlador de seguimiento de pared hasta recuperar anchura libre suficiente.

Fórmulas concretas usadas

$$a_i = \text{clamp}\left(\frac{d_{\text{máx}} - d_i}{d_{\text{máx}} - d_{\text{mín}}}, 0, 1\right), \quad v_L = \sum_i G_{L,i} a_i,\ v_R = \sum_i G_{R,i} a_i,$$

$$v = \alpha(d_f)\, v_{\text{field}} + (1 - \alpha(d_f))\, \frac{v_L + v_R}{2}, \quad w = \alpha(d_f)\, w_{\text{field}} + (1 - \alpha(d_f))\, k_\Delta (v_R - v_L),$$

con $\alpha(d_f) = \text{clamp}\left(\frac{d_f - d_{\text{th}}}{d_{\text{max}} - d_{\text{th}}}, 0, 1\right)$ y d_f la distancia frontal mínima medida por sensores.

Parámetros iniciales recomendados para pruebas

- Ganancias Braitenberg: valores iniciales en $[0{,}2, 1{,}0]$ según sensor (frontales mayores).
- Filtrado exponencial β: $0{,}85$–$0{,}95$.
- Límites: $v_{\text{máx}} = 0{,}15$ m/s, $w_{\text{máx}} = 1{,}0$ rad/s.
- Mezcla adaptativa: $d_{\text{th}} = 0{,}25$ m, $d_{\text{máx}} = 0{,}6$ m.
- Limitación de aceleración: $a_{\text{máx}} = 0{,}3$ m/s^2.

Evaluación experimental

Registrar para cada variante: tiempo a meta, distancia mínima a obstáculos, número de estancamientos y número de colisiones/rozamientos. Presentar resultados en tablas y gráficas comparativas y justificar la elección final (por ejemplo, Variante B con α adaptativa suele ofrecer mejor compromiso entre seguridad y eficiencia).
En la Figura 29 se observa que el robot se ha agregado correctamente a la simulación.

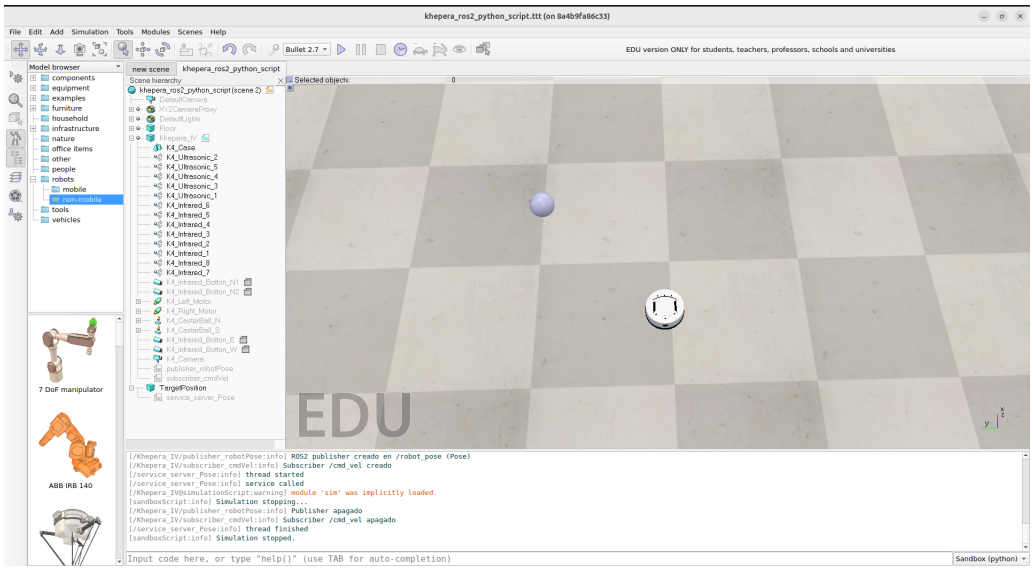

Figura 29: Robot *Khepera IV* y objeto que representa el punto de destino agregados a *Coppelia Sim*.

3. Iniciar la simulación y observar el comportamiento del robot. ¿A qué se debe este comportamiento?

4. Abrir el script de programación del robot y observar los valores de las velocidades izquierda y derecha.

El robot se mueve dibujando una trayectoria en arco de circunferencia. Haciendo doble click en el script por defecto del robot, Figura 30, se abre el script que se presenta en el Código 3. Este comportamiento se debe a las velocidades angulares ($w_L,\ w_R$) asignadas a cada motor: 3 rad/s al izquierdo y 5 rad/s al derecho, respectivamente. Por tanto, el motor izquierdo gira más lento que el derecho y el robot realiza una trayectoria en arco de circunferencia de radio $R = 0{,}176$ metros, que se obtiene mediante la Ecuación (1) conociendo la distancia entre los centros de las ruedas (*axle track*), $L = 0{,}088$ metros [**kheperaiv_manual_rev3**].

$$R = \frac{L}{2} \cdot \frac{\omega_R + \omega_L}{\omega_R - \omega_L} \tag{23}$$

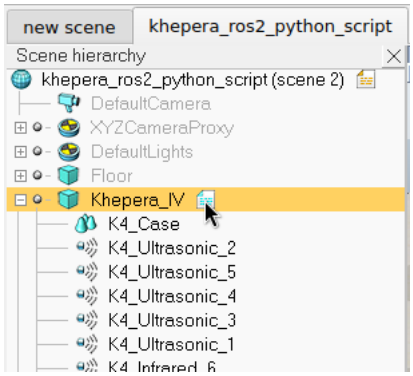

Figura 30: Script por defecto del robot.

5. Agregar a la simulación un objeto que represente el punto de destino del robot.

Se ha añadido una esfera con las propiedades descritas en la Figura 31 y en la posición $(x,\ y,\ z) = (0{,}75,\ 1{,}0,\ 0{,}05)$, como se observa en la Figura 29.

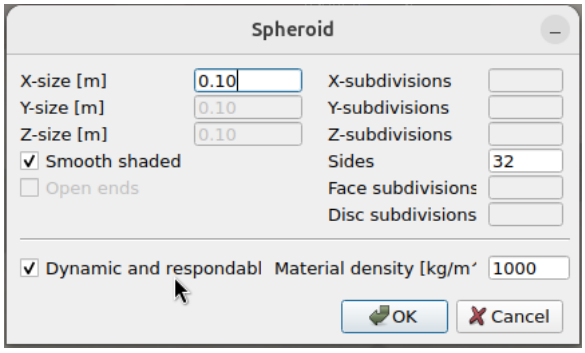

Figura 31: Propiedades de la esfera añadida como punto de destino.

6. Con las coordenadas del punto de destino y las del robot, calcular las velocidades angular y lineal para controlar su posición.

7. Obtener las velocidades que se envían al robot (izquierda y derecha) como resultado del modelo diferencial. Sugerencia: se puede comenzar solamente controlando la velocidad angular del robot a través el ángulo orientación hacia el punto y manteniendo la velocidad lineal constante, por medio de un control proporcional (P) sencillo. (*v*=cte, *w*=K_p*(error ángulo)).

8. Implementar una ley de control diferente a la proporcionada y comparar ambas usando los Índices de Comportamiento (*IAE*, *ISE*, *ITAE*, *ITSE*).

Dado que la implementación propuesta se ha realizado mediante *ROS2*, el primer paso es diseñar y definir el sistema de comunicación en *CoppeliaSim*. Para ello, se definen los siguiente requisitos:

- Se necesita obtener la posición del punto objetivo. Este destino es fijo, por lo que con obtener su posición al comienzo del programa es suficiente.
- Se necesita conocer la posición del robot en cada momento, ya que irá variando según las velocidades aplicadas.

- Se necesita accionar el robot con diferentes velocidades en cada momento, según el error calculado y la distancia restante.

En *ROS2* se dispone de tres vías de comunicación distintas: tópicos, servicios y acciones.

Un tópico es un canal de comunicación que permite el intercambio de mensajes de manera asíncrona, siguiendo el modelo publicador-suscriptor. Cada tópico está asociado a un tipo de mensaje específico. Los tópicos se utilizan, por ejemplo, para recibir información en tiempo real de los diferentes sensores del robot.

Un servicio es un mecanismo de comunicación síncrona basado en el modelo cliente-servidor que permite que un cliente envíe una petición a un servidor. El servidor procesa la solicitud y devuelve una respuesta al cliente. Son útiles para operaciones puntuales que requieren una respuesta inmediata, como pedir el estado de un sensor o sacar una foto. La comunicación mediante servicios involucra un mensaje de petición y otro de respuesta.

Una acción es una interfaz de comunicación asíncrona basada en tópicos y servicios diseñada para desempeñar operaciones largas y prolongadas en el tiempo que necesitan seguimiento. Para ejecutar una acción un cliente solicita a un servidor la ejecución, recibe retroalimentación periódica sobre el progreso a través de un tópico y tiene la opción de cancelar la tarea antes de que finalice. Son ideales para tareas como mover un robot a una posición determinada, donde es necesario monitorizar el avance y poder interrumpir la ejecución si es necesario. La comunicación de una acción involucra mensajes de objetivo, retroalimentación y resultado final.

Consecuentemente, el diseño de la arquitectura planteado, Figura 32, es el siguiente:

- Un tópico para publicar la posición del robot. El tópico se llama `/robot_pose` y utiliza el tipo de mensaje estándar `geometry_msgs/msg/Pose`. Este mensaje se compone de dos campos: un objeto *Point* (que se compone de tres números; x, y, z; en formato `float64`) y un objeto *Quaternion* (que se compone de cuatro números; x, y, z, w; en formato `float64`). Su documentación está accesible en el siguiente enlace: `https://docs.ros.org/en/jazzy/p/geometry_msgs/__message_definitions.html`.

  En *CoppeliaSim*, el robot tiene un script *Lua* llamado `publisher_robotPose` que ejerce de publicador de la posición, Código 4, mientras que el nodo o programa *ROS2* `khepera_controller`, externo a *CoppeliaSim*, Código 11, se suscribe al tópico.

- Un tópico para recibir los comandos de velocidad lineal y angular del robot. El tópico se llama `/cmd_vel` y utiliza el tipo de mensaje estándar `geometry_msgs/msg/Twist`. Este mensaje se compone de dos objetos *Vector3*: *linear* y *angular*. Ambos objetos se componen de tres campos cada uno; x, y, z; y representan la velocidad lineal o angular a aplicar en cada uno de los ejes. Su documentación también está accesible en el enlace anterior.

  En *CoppeliaSim*, el robot tiene un script *Lua* llamado `subscriber_cmdVel` que ejerce de suscriptor de la velocidad, Código 5, mientras que el nodo o programa *ROS2* `khepera_controller`, externo a *CoppeliaSim*, Código 11, publica en el tópico la velocidad deseada para comandar la velocidad al robot.

  Es importante destacar que el suscriptor implementado en *CoppeliaSim* transforma las velocidades lineales y angulares del robot comandadas en el tópico `/cmd_vel` en las velocidades individuales que debe ejecutar cada una de las ruedas del robot diferencial. La relación entre la velocidad lineal v y la velocidad angular ω del chasis con respecto a las velocidades angulares de las ruedas izquierda, ω_L, y derecha, ω_R viene dada por el

modelo cinemático, Ecuaciones 2 en caso de utilizar velocidades angulares en cada rueda, o Ecuaciones 3 en caso de utilizar velocidades lineales, v_L y v_R, en cada rueda, donde r es el radio de las ruedas y L es la distancia entre ruedas. En este caso, se utilizan las Ecuaciones 2 porque el robot en uso controla los motores mediante velocidades angulares.

$$v = \frac{r}{2}(\omega_R + \omega_L) \qquad \omega = \frac{r}{L}(\omega_R - \omega_L) \tag{24}$$

$$v = \frac{v_L + v_R}{2} \qquad \omega = \frac{v_R - v_L}{L/2} \tag{25}$$

Finalmente, despejando las velocidades angulares de cada rueda se obtiene la expresión necesaria para calcularlas, Ecuaciones 4.

$$\omega_L = \frac{v - \omega\frac{L}{2}}{r} \qquad \omega_R = \frac{v + \omega\frac{L}{2}}{r} \tag{26}$$

- Un servicio para solicitar la posición del objeto que representa el punto de destino en la trayectoria. El servicio se llama `/get_target_pose` y utiliza el tipo estándar `std_srvs/srv/Trigger`. Este tipo de servicio no recibe parámetros de entrada y devuelve una respuesta compuesta por dos campos: `success` (booleano) y `message` (cadena de texto). Su documentación está accesible en el siguiente enlace: `https://docs.ros.org/en/jazzy/p/std_srvs/index.html`.

  En *CoppeliaSim*, la esfera que representa el punto de destino dispone de un script *Lua* llamado `serviceServer_pose` que implementa el servidor del servicio, Código 6. Cuando el servicio es invocado desde el nodo *ROS2* externo `khepera_controller`, el servidor obtiene la posición y orientación del objeto. A continuación, empaqueta esta información en formato JSON dentro del campo `message` de la respuesta del servicio.

  Por su parte, el nodo ROS2 `khepera_controller`, Código 11, realiza la llamada al servicio `/get_target_pose` cada vez que necesita conocer la pose del punto de destino para actualizar la ley de control o generar la trayectoria correspondiente.

- Una acción para solicitar la ejecución de una trayectoria hacia un punto de destino. La acción se llama `/execute_trajectory` y utiliza el tipo personalizado `interfaces/action/ExecuteTrajectory`, Código 7. Esta acción está compuesta por tres estructuras: *Goal*, *Feedback* y *Result*. Su documentación está accesible en el siguiente enlace: `https://design.ros2.org/articles/actions.html`.

  La estructura *Goal* de esta acción contiene un campo de tipo `geometry_msgs/msg/Pose`. Si se le indica una posición, el robot inicia la trayectoria a esa posición. Si no se le indica, obtiene la posición de destino mediante el servicio `/get_target_pose`, solicitando de forma asíncrona la posición del punto de destino al servidor del servicio, evitando bloqueos en el *callback* de la acción. Como el movimiento se desarrolla en los ejes X e Y, no se tiene en cuenta el valor de Z (poner 0.0 por defecto).

  La estructura *Feedback* contiene información relevante sobre la evolución de la trayectoria en tiempo real. En concreto, incluye:

  - `distance`: distancia actual del robot al objetivo.
  - `e_theta`: error angular entre la orientación del robot y la dirección hacia el objetivo.
  - `v`: velocidad lineal que el controlador está aplicando.

- `w`: velocidad angular aplicada.
- `time_elapsed`: tiempo transcurrido desde el inicio de la acción.

Esta información se publica periódicamente mientras el robot avanza hacia el objetivo, permitiendo monitorizar la evolución del control en tiempo real.

La estructura *Result* contiene dos campos:

- `success`: booleano que indica si el robot ha alcanzado el objetivo.
- `message`: cadena de texto con información adicional sobre el resultado de la acción.

El nodo *ROS2* externo `khepera_controller`, Código 11, implementa el servidor de la acción. Cuando un cliente envía una petición a `/execute_trajectory`, el servidor:

1. Acepta la meta.
2. Solicita de forma asíncrona la posición del objetivo mediante el servicio `/get_target_pose`.
3. Una vez recibida la posición, inicia el bucle de control periódico.
4. Publica *feedback* en cada iteración.
5. Finaliza la acción cuando el robot alcanza el objetivo o si se produce una cancelación.

Para realizar la solicitud de la acción desde terminal, ejecutar el Código 8. En dicho código aparece tanto la llamada indicando una posición como sin indicarla, en la que es suficiente con establecer el valor de cualquiera de los campos a "nan" para que obtenga mediante el servicio la posición del objeto como punto de destino.

- Un nodo *ROS2* encargado de implementar la lógica de control del robot y coordinar los distintos mecanismos descritos anteriormente. Este nodo, llamado `khepera_controller`, Código 11, actúa como elemento central del sistema, ya que:

  - se suscribe al tópico `/robot_pose` para recibir la pose del robot,
  - publica comandos de velocidad en el tópico `/cmd_vel`,
  - invoca el servicio `/get_target_pose` para obtener la posición del objetivo,
  - y expone la acción `/execute_trajectory` para permitir que un cliente solicite la ejecución de una trayectoria hacia dicho objetivo.

  El nodo implementa un servidor de la acción `ExecuteTrajectory`. Cuando un cliente envía una petición, el nodo acepta la meta y solicita de forma asíncrona la posición del objetivo mediante el servicio `/get_target_pose`. Este diseño evita bloquear el *callback* de la acción y permite que el nodo continúe procesando otros eventos mientras espera la respuesta del servicio.

  Una vez recibida la posición del objetivo, el nodo activa un bucle de control hasta alcanzar el objetivo. En cada iteración del bucle:

  1. Se obtiene la pose actual del robot a partir del último mensaje recibido en el tópico `/robot_pose`.
  2. Se calcula el error de posición y orientación respecto al objetivo.
  3. Se aplica la ley de control seleccionada, generando las velocidades lineal v y angular ω.

4. Se publican dichas velocidades en el tópico `/cmd_vel`.
5. Se envía al cliente de la acción un mensaje de *feedback* con información sobre la evolución de la trayectoria: distancia al objetivo, error angular, velocidades aplicadas y tiempo transcurrido.

El nodo considera que el objetivo ha sido alcanzado cuando la distancia al punto de destino es inferior a un umbral predefinido (por ejemplo, 5 centímetros).

Además, este nodo incorpora un módulo específico para la evaluación cuantitativa del rendimiento del controlador. Este módulo, implementado en la clase `MetricsLogger`, Códigos 15 y 16, registra en cada iteración del bucle de control tanto las variables relevantes del estado del robot (posición, orientación, distancia al objetivo y errores de control) como las velocidades lineal y angular aplicadas. A partir de estos datos, el sistema calcula varias métricas clásicas de evaluación de control, tales como el *Integral of Absolute Error* (*IAE*), el *Integral of Squared Error* (*ISE*), el *Integral of Time-weighted Absolute Error* (*ITAE*) y el *Integral of Time-weighted Squared Error* (*ITSE*). Estas métricas permiten analizar la calidad de la trayectoria ejecutada y comparar el comportamiento de distintos parámetros o estrategias de control.

Al finalizar la acción, el nodo almacena toda esa información en un fichero *CSV*, incluyendo una fila por cada instante de tiempo con las variables medidas y calculadas, seguida de un bloque final con los valores agregados de las métricas y los parámetros del controlador utilizados (`kp`, `vmax`, `wmax` y el tipo de controlador seleccionado).

Por último, mencionar que para facilitar la depuración y modularidad del código, en lugar de implementar todas las responsabilidades en el propio `khepera_controller`, se han definido tanto las cabeceras como las funciones en distintos módulos.

El módulo `service_utils.hpp/.cpp` contiene las funciones auxiliares para realizar la llamada asíncrona al servicio `/get_target_pose`, Códigos 12 y 13, el módulo `pose_utils.hpp` proporciona utilidades para extraer y manipular la pose del robot (posición y orientación) a partir de los mensajes recibidos, Código 14, y el módulo `metrics_utils.hpp/.cpp` implementa el sistema de evaluación y registro de métricas, Códigos 15 y 16.

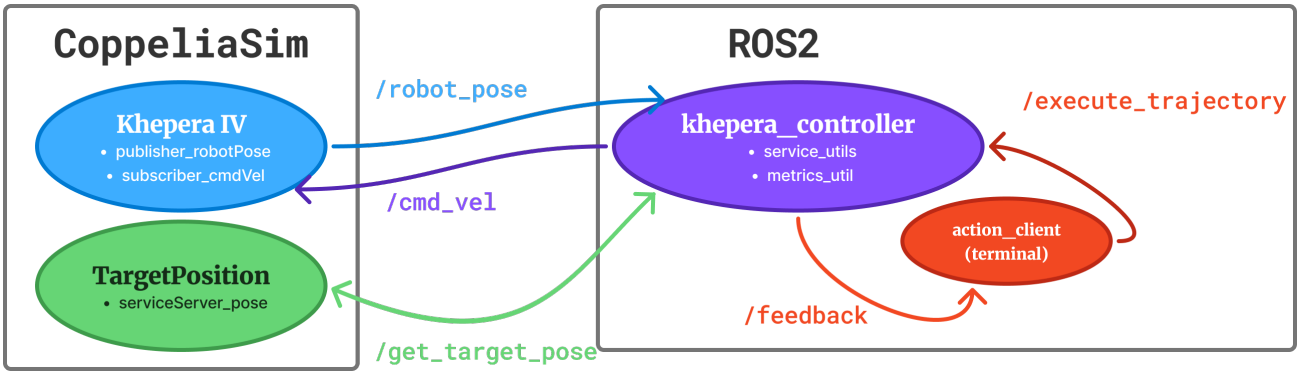

Figura 32: Esquema de la arquitectura propuesta.

Controladores implementados

En esta práctica se implementan y comparan dos controladores cinemáticos para un robot diferencial basados en el concepto de *Instantaneous Center of Curvature* (ICC). El primero es

un controlador proporcional reactivo (control-P) hacia un objetivo, mientras que el segundo es un controlador geométrico basado en curvatura, equivalente al enfoque *Pure Pursuit* puntual. Ambos enfoques están ampliamente descritos en la literatura clásica de control de robots móviles [**choset2005**, **siegwart2011**, **siciliano2009**].

Geometría del problema

Sea la posición del robot (x_r, y_r) y la del objetivo (x_t, y_t), se define:

$$d = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2} \tag{27}$$

$$\theta_d = \text{atan2}(y_t - y_r,\ x_t - x_r) \tag{28}$$

$$e_\theta = \theta_d - \theta_r \tag{29}$$

donde d es la distancia al objetivo, e_θ el error de orientación del robot respecto del objetivo, θ_d el ángulo absoluto definido en el marco global de la orientación de la línea recta que une la posición del robot con la del objetivo y θ_r la orientación actual del robot. De este modo, θ_d indica la dirección hacia la que debería orientarse el robot para apuntar directamente al objetivo, mientras que el error angular $e_\theta = \theta_d - \theta_r$ mide la diferencia entre dicha orientación deseada y la orientación actual del robot. Esta formulación es estándar en control cinemático de robots diferenciales [**otero2005**, **morin2016**].

En la Figura 33 se presenta el esquema de un robot móvil diferencial con ruedas.

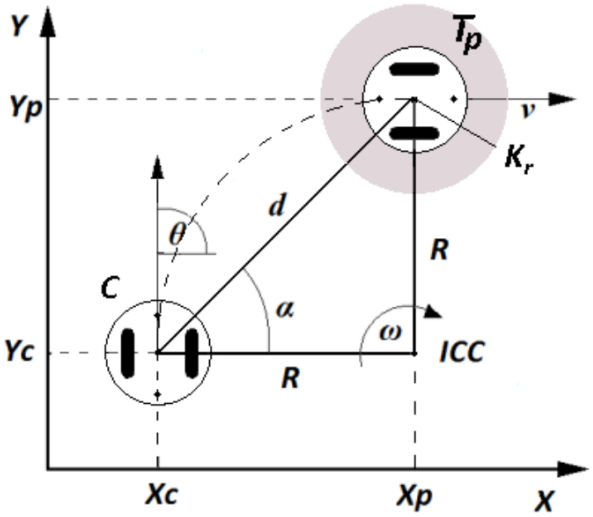

Figura 33: Esquema de un robot móvil diferencial con ruedas [**FARIAS20179150**].

1. Controlador proporcional reactivo

El controlador reactivo utiliza una ley proporcional para la velocidad lineal,

$$v = K_p \cdot d\ , \tag{30}$$

saturada por la velocidad máxima del robot:

$$v = \text{mín}(v,\ v_{\text{máx}})\ . \tag{31}$$

La velocidad angular se calcula como:

$$\omega = \omega_{\text{máx}} \cdot \sin(e_\theta) \tag{32}$$

Este tipo de controlador proporcional basado en errores de pose es ampliamente utilizado en robots uniciclo y se describe en [**choset2005**, **siegwart2011**].

2. Controlador geométrico basado en curvatura o *Pure Pursuit* puntual

El controlador geométrico impone que el robot siga un arco de circunferencia que conecta su pose actual con el objetivo. El radio del arco viene dado por:

$$R = \frac{d}{2\sin(e_\theta)} \tag{33}$$

La velocidad angular se obtiene mediante la relación cinemática:

$$\omega = \frac{v}{R} \tag{34}$$

Este enfoque corresponde a la formulación puntual del algoritmo *Pure Pursuit*, descrito originalmente en [**coulter1992**] y ampliamente analizado en [**otero2005**, **morin2016**, **corke2017**]. A diferencia del controlador reactivo, la forma de la trayectoria depende únicamente de la geometría del problema.

Evaluación

Para comparar ambos controladores se han ejecutado trayectorias desde la posición inicial del robot, ($x = 0{,}27887,\ y = 0{,}525$), hasta la posición de destino, ($x = 0{,}75,\ y = 1{,}0$), estableciendo la velocidad máxima del robot a $1m/s$, [**kheperaiv_manual_rev3**] y realizando un barrido del valor de K_p, comenzando en $0{,}1$ y aumentando en $0{,}05$ hasta $0{,}6$, ya que con una ganancia superior la aceleración del robot es excesiva y provoca el volcado del mismo. El control de la aceleración queda fuera del alcance de esta tarea.

La evaluación de los controladores se basa en el cálculo y comparación de las siguientes métricas estándar en control:

- **Integral del Error Absoluto (*IAE*)**: mide el error acumulado a lo largo del tiempo, proporcionando una valoración global del desempeño sin penalizar en exceso los errores grandes. Es adecuada cuando se desea evaluar el comportamiento medio del sistema o cuando los errores puntuales no son críticos.

$$\textit{IAE} = \int_0^T |e(t)|\, dt \tag{35}$$

- **Integral del Error Cuadrático (*ISE*)**: penaliza de forma más severa los errores grandes, siendo útil para evaluar controladores que deben minimizar desviaciones significativas o evitar comportamientos agresivos. Es apropiada cuando los errores grandes son especialmente indeseables.

$$\textit{ISE} = \int_0^T e(t)^2\, dt \tag{36}$$

- **Integral del Error Absoluto Ponderado por el Tiempo (*ITAE*)**: penaliza especialmente los errores que persisten en el tiempo, favoreciendo respuestas rápidas y con menor cola temporal. Se utiliza cuando el objetivo es minimizar el tiempo de asentamiento y mejorar la rapidez de convergencia.

$$\textit{ITAE} = \int_0^T t\,|e(t)|\,dt \tag{37}$$

- **Integral del Error Cuadrático Ponderado por el Tiempo (*ITSE*)**: combina la penalización de errores grandes con la penalización de errores tardíos, siendo útil para evaluar estabilidad y convergencia en sistemas no lineales. Es adecuada cuando interesa evitar tanto errores grandes como oscilaciones prolongadas.

$$\textit{ITSE} = \int_0^T t\,e(t)^2\,dt \tag{38}$$

En la práctica, estas métricas deben implementarse de manera discreta, por lo que dado el periodo de muestreo Δt, las métricas se aproximan mediante sumas:

$$\textit{IAE} \approx \sum_{k=0}^{N} |e_k|\,\Delta t \tag{39}$$

$$\textit{ISE} \approx \sum_{k=0}^{N} e_k^2\,\Delta t \tag{40}$$

$$\textit{ITAE} \approx \sum_{k=0}^{N} t_k\,|e_k|\,\Delta t \tag{41}$$

$$\textit{ITSE} \approx \sum_{k=0}^{N} t_k\,e_k^2\,\Delta t \tag{42}$$

Estas métricas permiten comparar objetivamente el rendimiento de ambos controladores bajo diferentes valores de K_p.

Registro de datos

En cada iteración del controlador se almacena el tiempo t, la pose del robot (x_r, y_r, θ_r), la pose del objetivo (x_t, y_t), error de distancia d, error de orientación e_θ y velocidades v y ω.

Al finalizar la acción, el sistema añade al fichero *CSV* las métricas globales calculadas (*IAE*, *ISE*, *ITAE*, *ITSE*) y los parámetros utilizados (K_p, $v_{\text{máx}}$, $\omega_{\text{máx}}$ y tipo de controlador). Este registro permite realizar comparativas rigurosas mediante tablas y gráficas.

Resultados

En las Figuras 34 a 44 se presentan las gráficas que dibujan las distintas señales medidas durante la ejecución de la trayectoria del primer controlador.

En las Figuras 45 a 55 se presentan las gráficas que dibujan las distintas señales medidas durante la ejecución de la trayectoria del segundo controlador.

En la Tabla 3 se muestran las cuatro métricas de evaluación calculadas para cada controlador y cada configuración, donde se ha coloreado en amarillo los valores más bajos obtenidos para cada métrica.

Estos resultados evidencian que el controlador proporcional reactivo mantiene un desempeño consistente a lo largo de distintos valores de K_p, con reducciones progresivas en todas las métricas al aumentar la ganancia, por lo que disminuyen tanto los errores acumulados como los errores tardíos y los picos de error, reflejando un comportamiento estable y uniforme.

El controlador geométrico basado en curvatura, por su parte, presenta inicialmente valores de error mayores, especialmente en *IAE* e *ITAE*, lo que refleja errores acumulados y tardíos superiores para valores bajos de K_p. Sin embargo, a medida que la ganancia aumenta, la reducción de todas las métricas es más pronunciada que en el controlador proporcional. Esta mayor pendiente de mejora indica que, aunque al principio su desempeño es inferior, el controlador geométrico alcanza rápidamente un rendimiento superior. De hecho, para $K_p = 0{,}5$, logra los menores valores en todas las métricas, lo que demuestra que minimiza eficazmente todos los errores, superando al controlador proporcional reactivo en precisión y rapidez de convergencia.

Conclusiones

La comparación evidencia que el controlador proporcional reactivo es más estable y uniforme a lo largo de los valores de la ganancia K_p, mientras que el controlador geométrico tiene un desempeño más sensible a la ganancia, pero capaz de superar al reactivo cuando la ganancia alcanza valores óptimos. Esta situación resalta la importancia de ajustar correctamente las ganancias para aprovechar la mayor capacidad de seguimiento del controlador geométrico, especialmente cuando se busca minimizar errores tardíos y grandes y maximizar la precisión global en la trayectoria.

		0.1	**0.15**	**0.2**	**0.25**	**0.3**	**0.35**	**0.4**	**0.45**	**0.5**	**0.55**	**0.6**
Cont. prop. reactivo	***IAE***	2.83	1.91	1.46	1.18	1.03	0.92	0.81	0.74	0.76	0.86	0.85
	ISE	1.05	0.73	0.56	0.46	0.42	0.38	0.33	0.31	0.33	0.39	0.43
	ITAE	9.99	4.47	2.58	1.66	1.21	0.97	0.74	0.61	0.63	0.75	0.66
	ITSE	2.29	1.06	0.61	0.41	0.31	0.25	0.19	0.17	0.18	0.24	0.24
Cont. geom.	***IAE***	3.16	2.16	1.64	1.28	1.05	0.95	0.76	0.77	0.7	0.75	0.88
	ISE	1.21	0.83	0.64	0.51	0.42	0.38	0.31	0.32	0.3	0.34	0.45
	ITAE	11.94	5.57	3.17	1.92	1.29	1.03	0.67	0.67	0.55	0.59	0.7
	ITSE	2.9	1.36	0.79	0.48	0.33	0.27	0.17	0.18	0.15	0.18	0.26

Tabla 3: Evaluación de los controladores con las cuatro métricas propuestas para distintos valores de K_p.

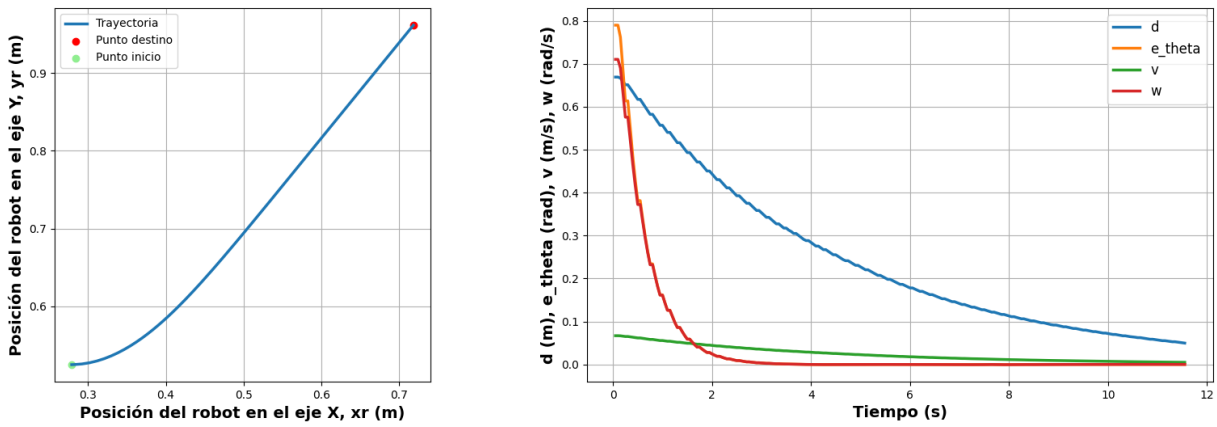

Figura 34: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}1$.

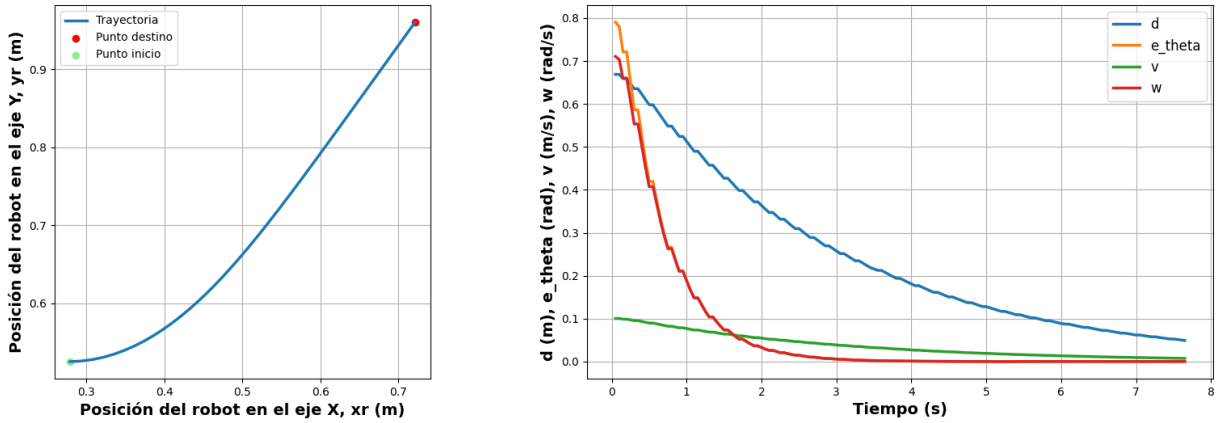

Figura 35: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}15$.

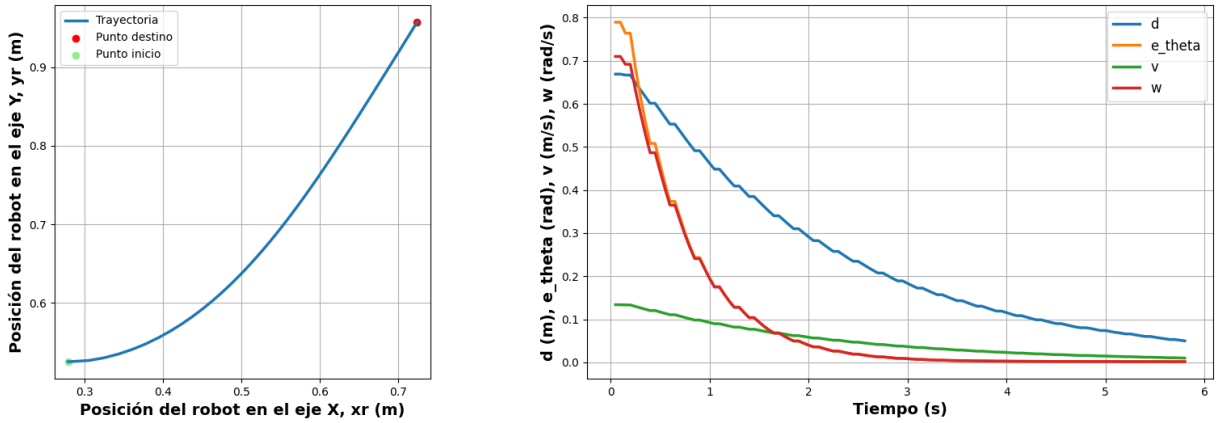

Figura 36: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}2$.

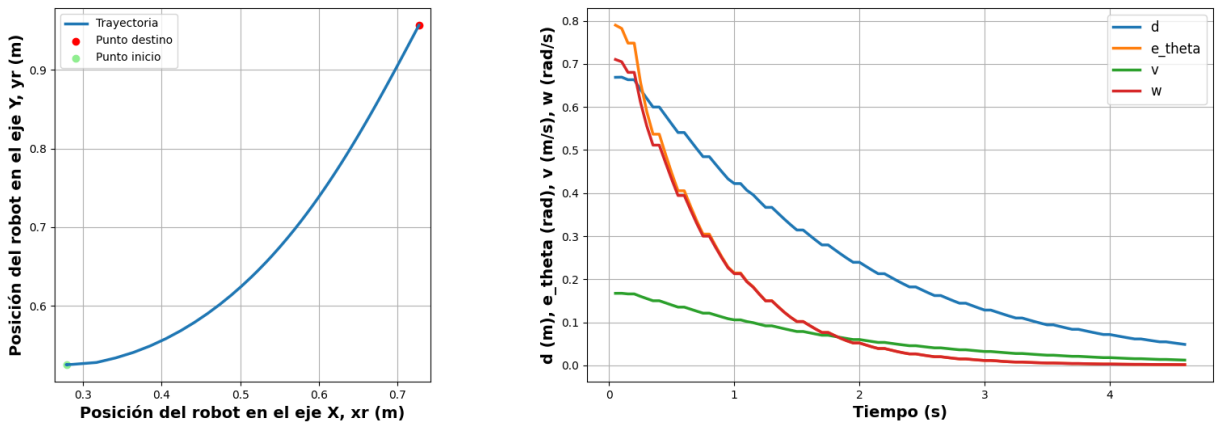

Figura 37: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}25$.

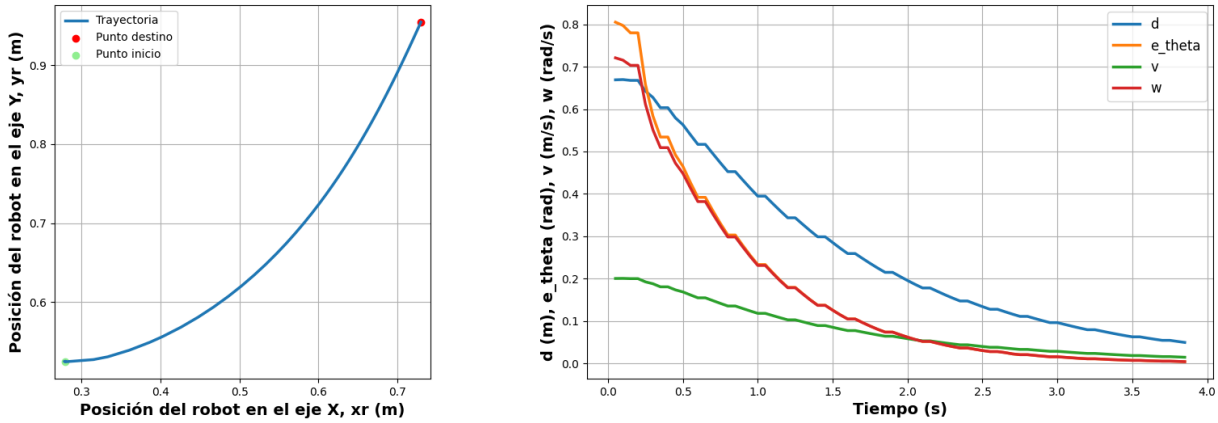

Figura 38: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}3$.

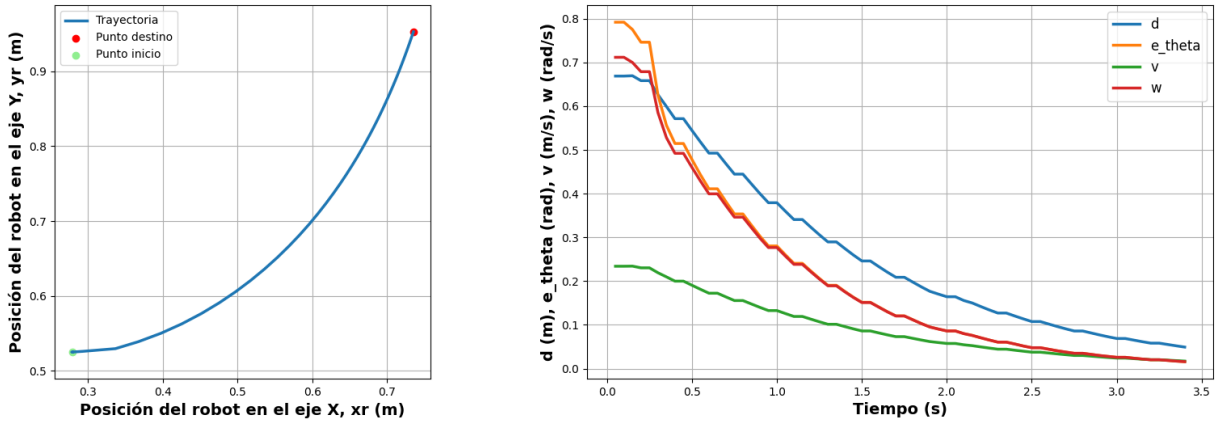

Figura 39: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}35$.

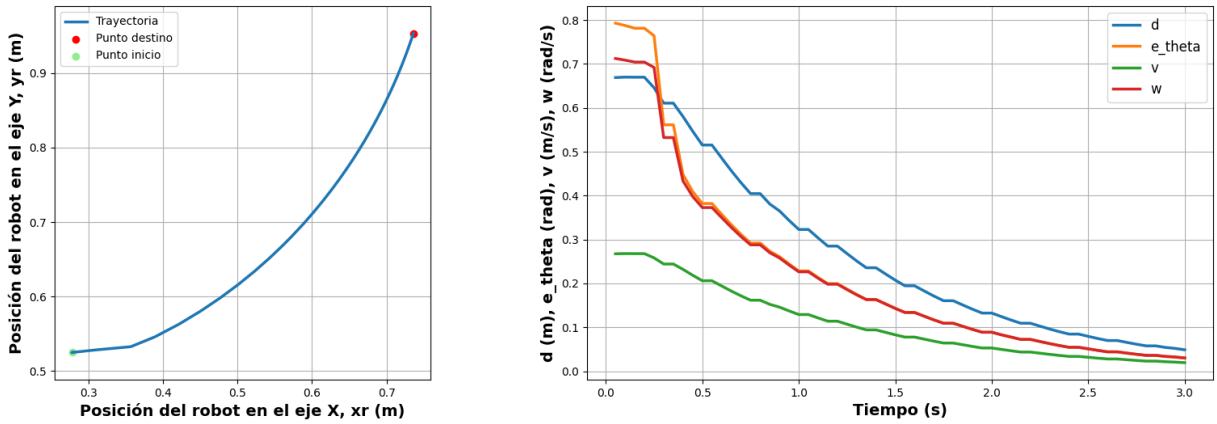

Figura 40: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}4$.

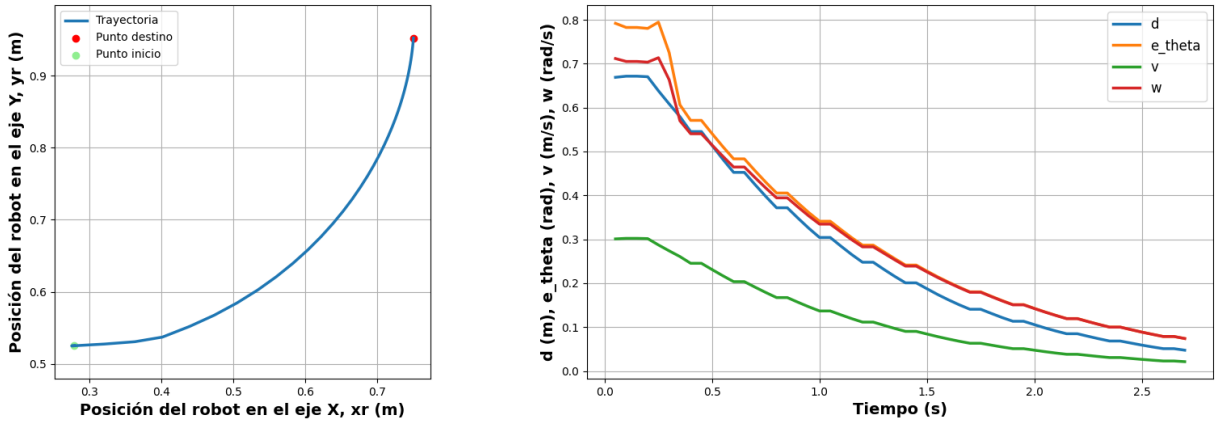

Figura 41: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}45$.

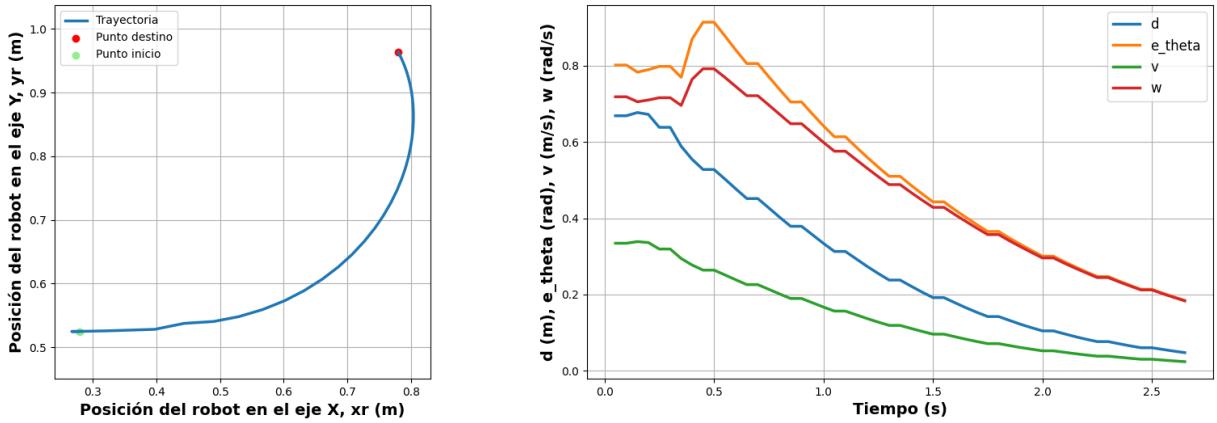

Figura 42: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}5$.

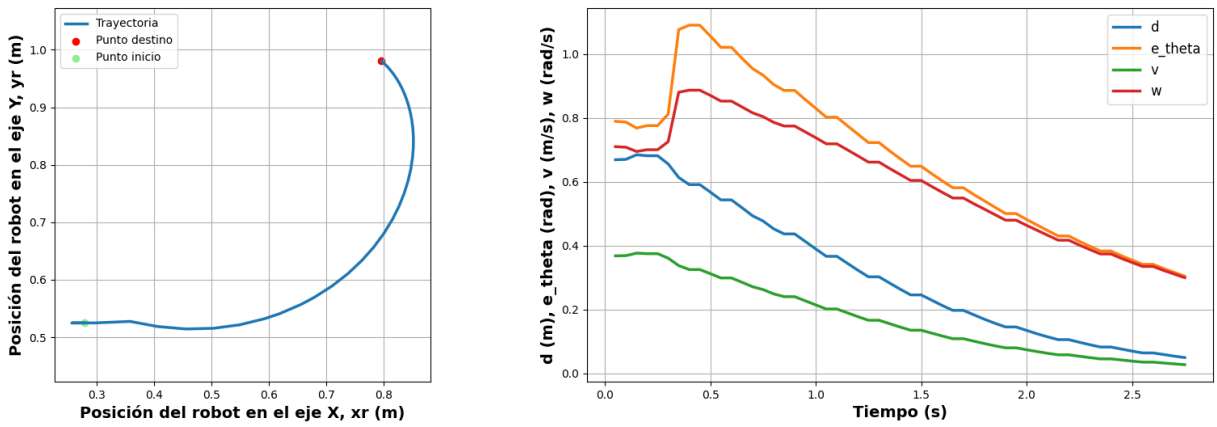

Figura 43: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}55$.

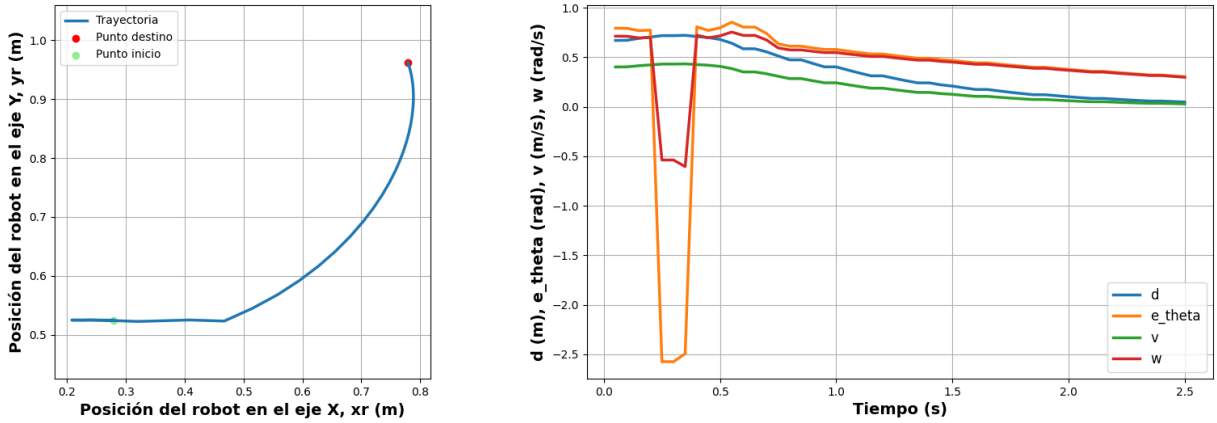

Figura 44: Trayectoria y señales de velocidades y error con el controlador reactivo y $K_p = 0{,}6$.

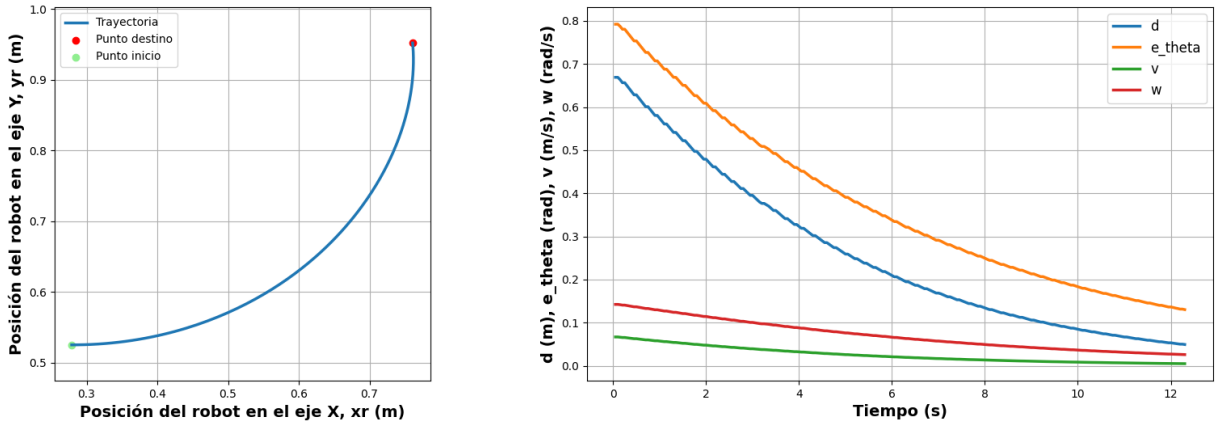

Figura 45: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}1$.

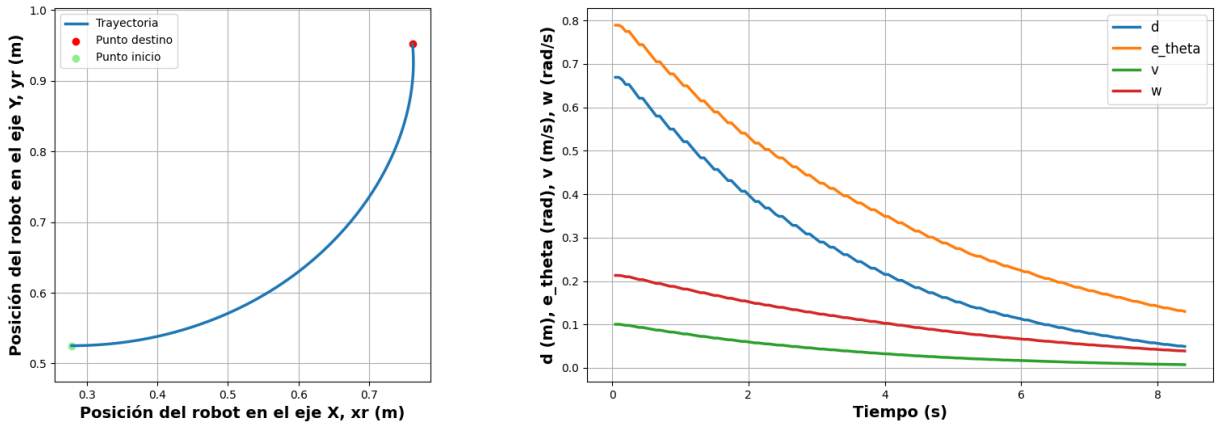

Figura 46: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}15$.

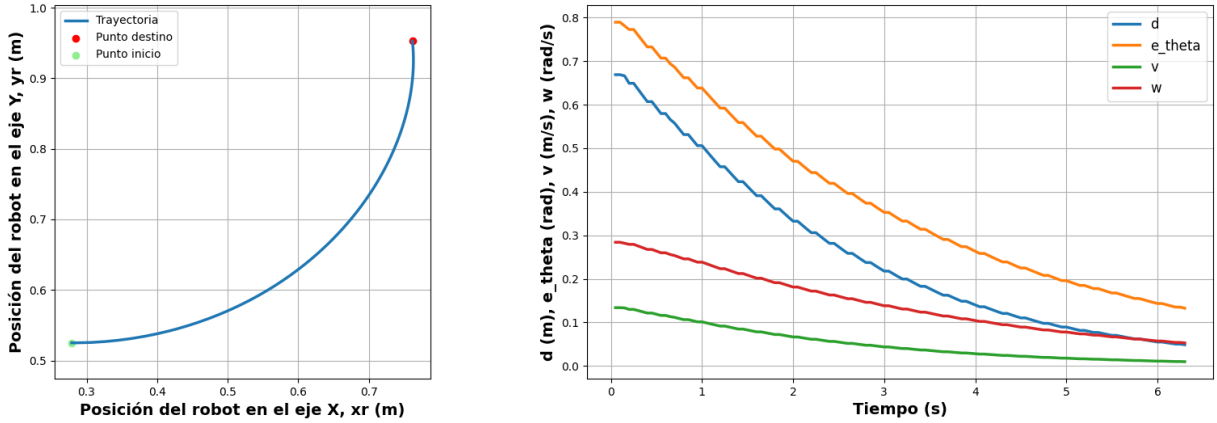

Figura 47: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}2$.

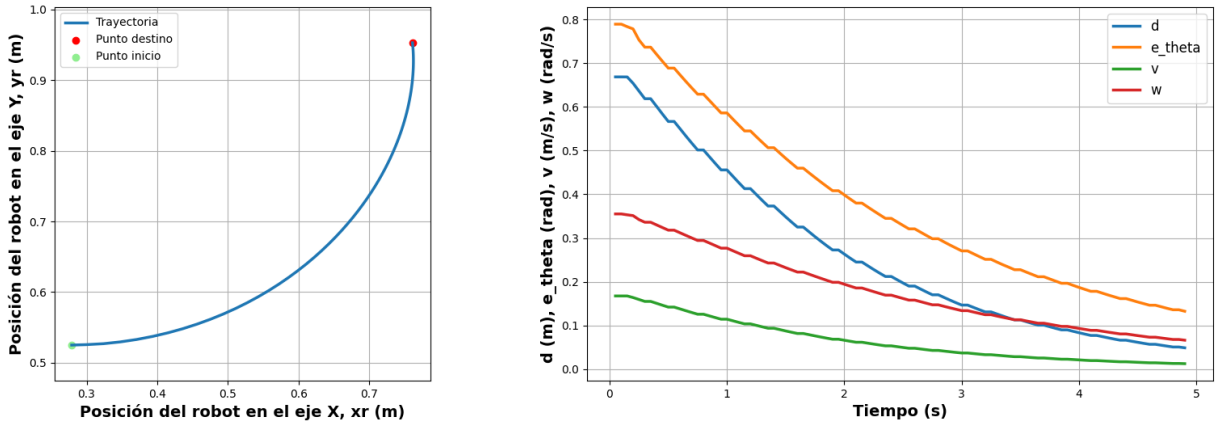

Figura 48: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}25$.

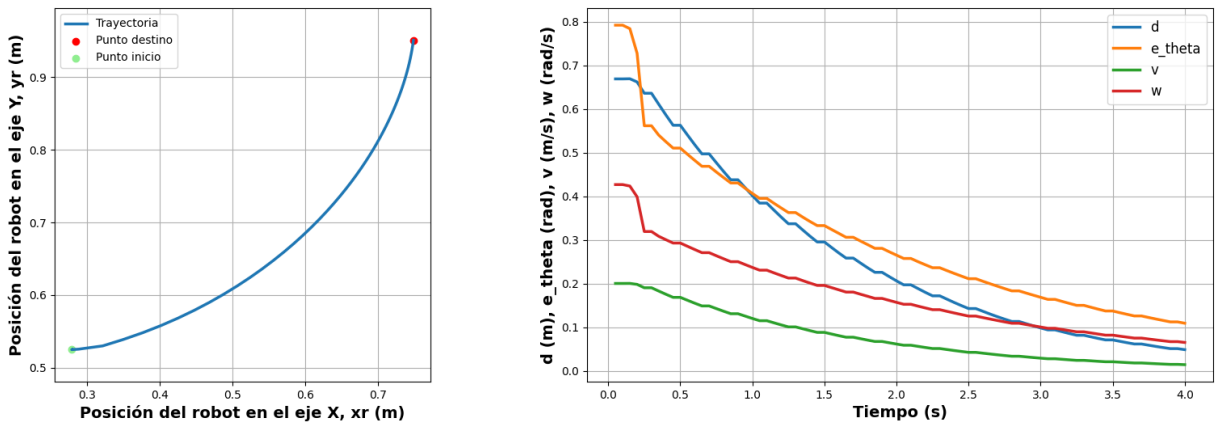

Figura 49: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}3$.

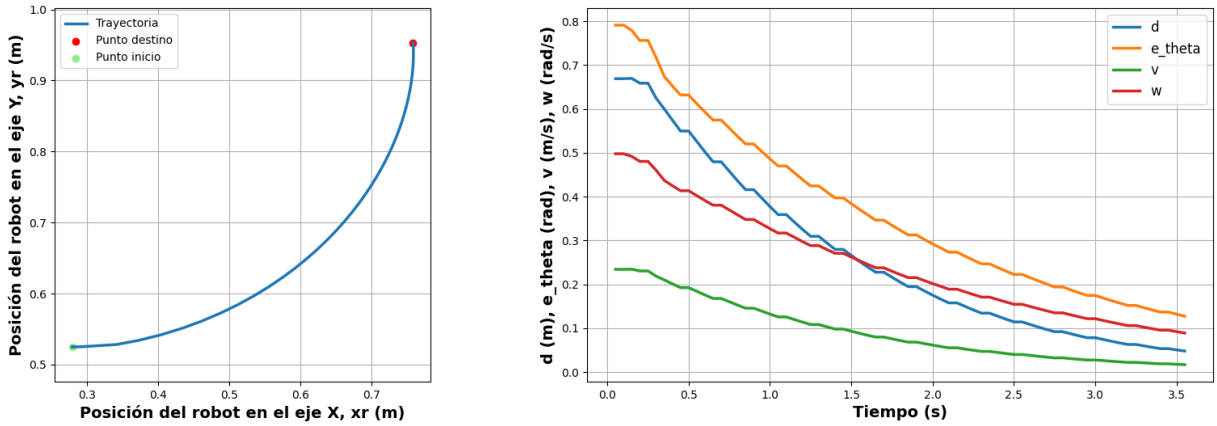

Figura 50: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}35$.

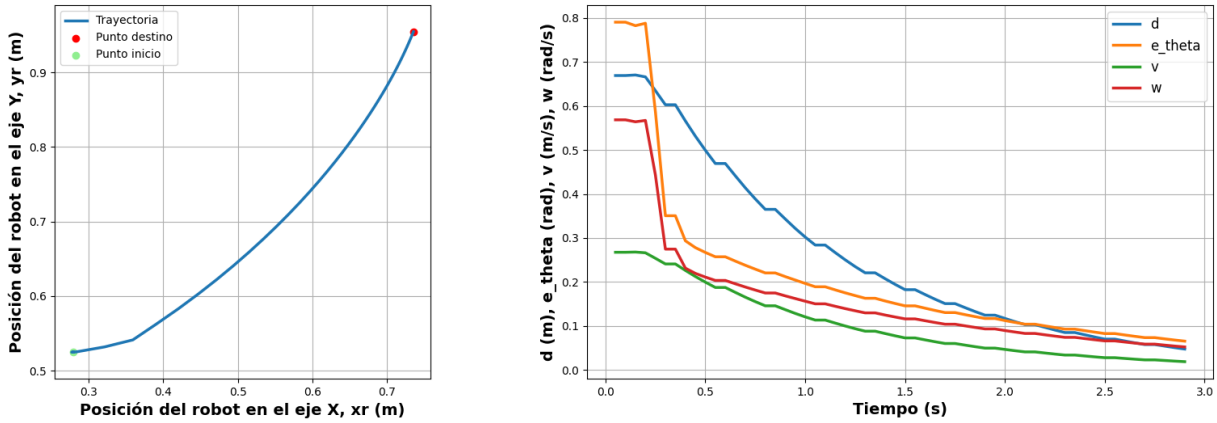

Figura 51: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}4$.

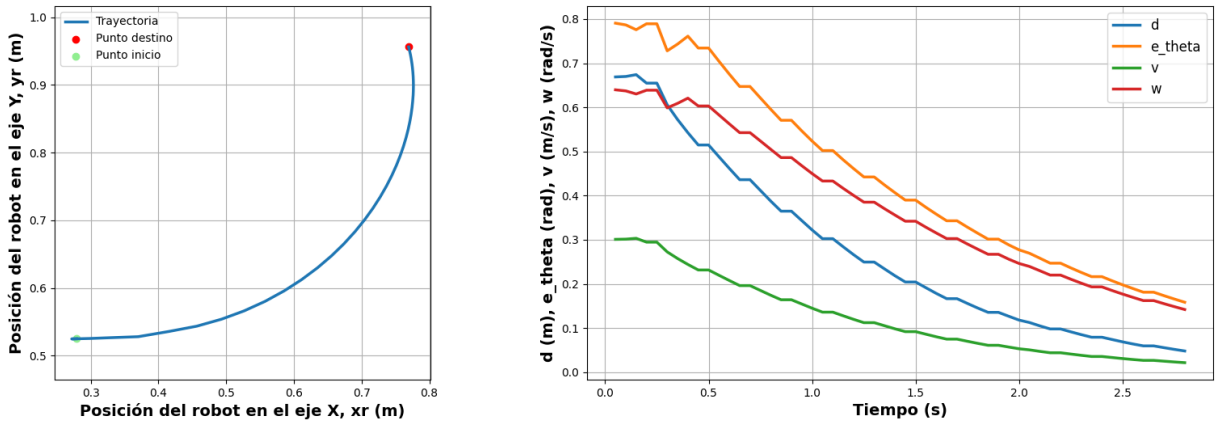

Figura 52: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}45$.

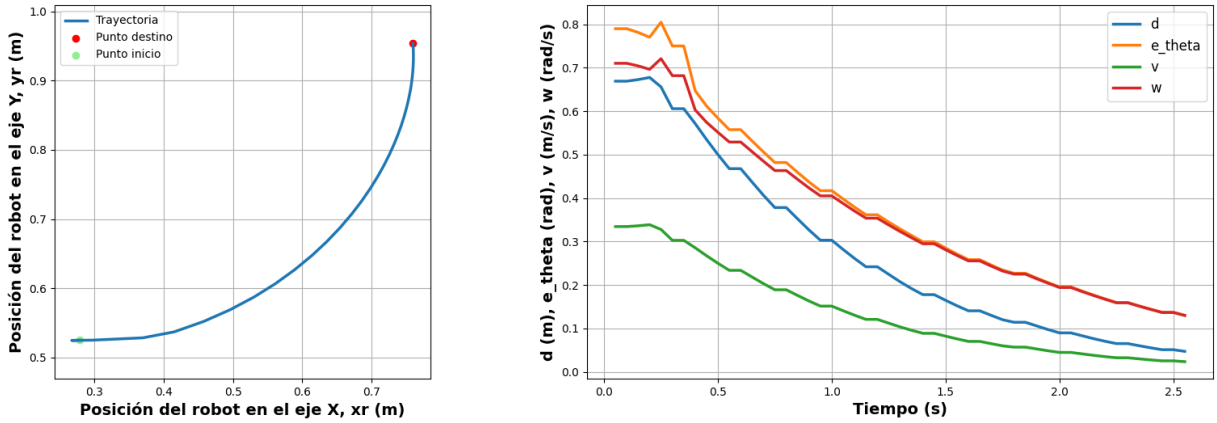

Figura 53: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}5$.

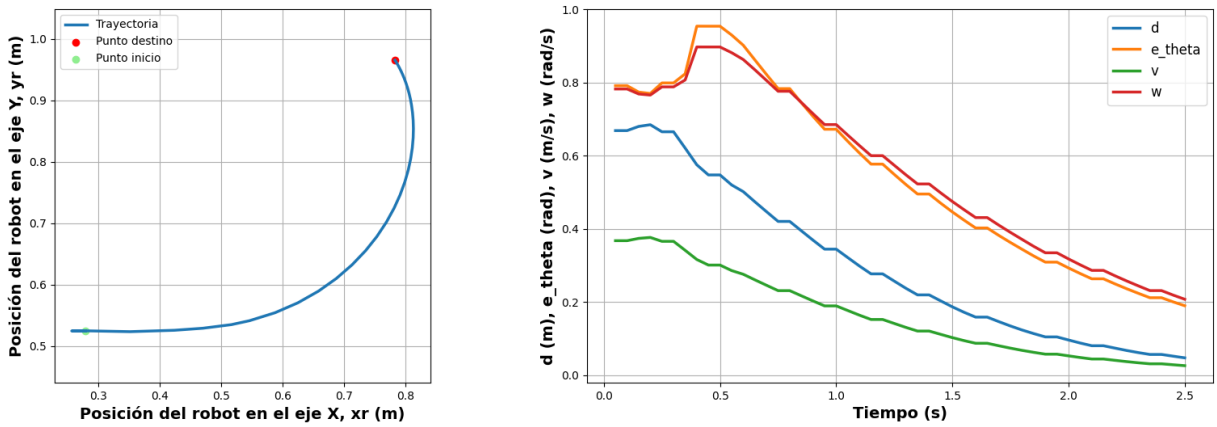

Figura 54: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}55$.

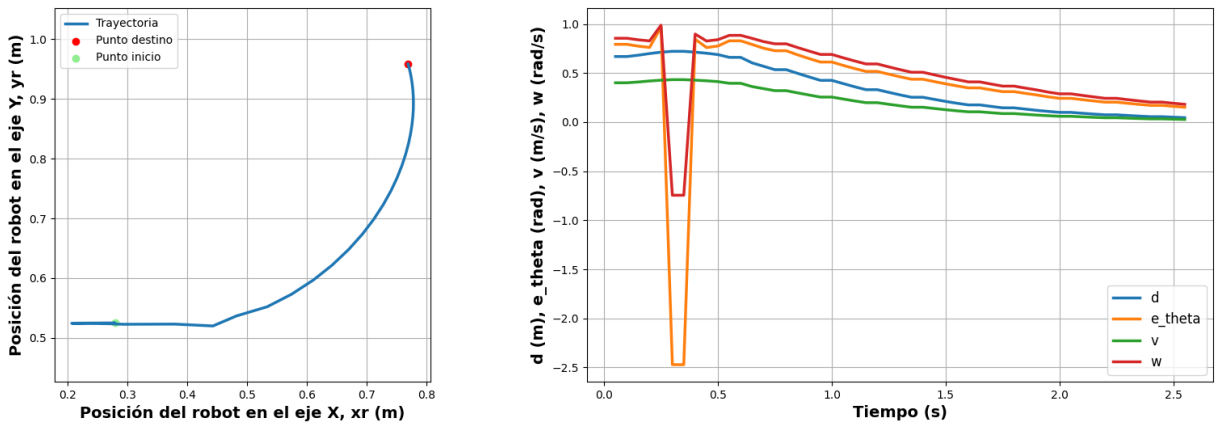

Figura 55: Trayectoria y señales de velocidades y error con el controlador geométrico y $K_p = 0{,}6$.

ANEXO I: código

Dockerfile

Código 1: *Dockerfile*.

```
FROM osrf/ros:jazzy-desktop-full

# --- Dependencias necesarias para CoppeliaSim ---
RUN apt-get update && apt-get install -y \
    libglu1-mesa \
    libxrender1 \
    libxi6 \
    libxrandr2 \
    libxcursor1 \
    libxinerama1 \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# Instalar módulos Python necesarios para CoppeliaSim
RUN apt-get update && apt-get install -y python3-pip && \
    pip3 install --break-system-packages pyzmq cbor2


# --- Crear directorio para CoppeliaSim ---
WORKDIR /opt

# --- Copiar y descomprimir CoppeliaSim ---
COPY CoppeliaSim_Edu_V4_10_0_rev0_Ubuntu24_04.tar.xz .
RUN tar -xf CoppeliaSim_Edu_V4_10_0_rev0_Ubuntu24_04.tar.xz && \
    rm CoppeliaSim_Edu_V4_10_0_rev0_Ubuntu24_04.tar.xz

# --- Copiar modelos del robot ---
COPY robot_description/* \
    /opt/CoppeliaSim_Edu_V4_10_0_rev0_Ubuntu24_04/models/robots/mobile/

# --- Copiar entrypoint de ROS ---
COPY ./ros_entrypoint.sh /ros_entrypoint.sh
RUN chmod +x /ros_entrypoint.sh

ENTRYPOINT ["/ros_entrypoint.sh"]
CMD ["/bin/bash"]
```

run.sh

Código 2: *run.sh*.

```
xhost +local:*
docker run -e DISPLAY=$DISPLAY \
           -e USER=$USER \
           -e QT_X11_NO_MITSHM=1 \
           -e NVIDIA_VISIBLE_DEVICES=all \
           -e NVIDIA_DRIVER_CAPABILITIES=all \
           -v /tmp/.X11-unix/:/tmp/.X11-unix/ \
           -v ./bibliografia/:/home/$USER/pcr_misc/bibliografia \
           -v ./robot_description/:/home/$USER/pcr_misc/robot_description \
           -v ./tareas/:/home/$USER/pcr_misc/tareas \
           -it \
           --rm \
           --gpus all \
           --runtime=nvidia \
           --name coppelia_sim \
           coppelia_sim_ros2:jazzy
```

Script por defecto del robot

Código 3: Script por defecto del robot.

```
------------Llamando a los elementos-----------------

    bodyElements=sim.getObjectHandle('Khepera_IV')
    rightMotor=sim.getObjectHandle('K4_Right_Motor')
    leftMotor=sim.getObjectHandle('K4_Left_Motor')

    -----------------------------------------------------

    -------Separando a Objetivo del Robot Khepera IV------

    -----------------------------------------------------

    ----------Calling camera----------------------

    frontCam=sim.getObjectHandle('K4_Camera')
    frontView=sim.floatingViewAdd(0.7,0.9,0.2,0.2,0)
    sim.adjustView(frontView,frontCam,64)

    -----------------------------------------------------

     while (true) do

        rv=5
        lv=3

        sim.setJointTargetVelocity(leftMotor,lv) --Cm/s a m/s
        sim.setJointTargetVelocity(rightMotor,rv)


end
```

Publicador de la posición del robot

Código 4: Publicador de la posición del robot.

```
function sysCall_init()
    sim = require'sim'
    simROS2 = require'simROS2'

    -- Obtener el objeto al que está asociado este script
    robot = sim.getObject('.')

    sim.addLog(sim.verbosity_scriptinfos,
        string.format("Lua script iniciado. Handle del robot: %d", robot))
end


function sysCall_thread()
    sim.addLog(sim.verbosity_scriptinfos, "Thread iniciado (publisher)")

    -- Crear publicador ROS2 para geometry_msgs/msg/Pose
    pub = simROS2.createPublisher(
        '/robot_pose',
        'geometry_msgs/msg/Pose'
    )

    sim.addLog(sim.verbosity_scriptinfos,
        "ROS2 publisher creado en /robot_pose (Pose)")

    -- Bucle de publicación (igual que el ejemplo oficial)
    while true do
        -- Obtener posición y orientación absolutas
        local pos = sim.getObjectPosition(robot, -1)
        local quat = sim.getObjectQuaternion(robot, -1)

        -- Crear mensaje Pose
        local msg = {
```

```
            position = {
                x = pos[1],
                y = pos[2],
                z = pos[3]
            },
            orientation = {
                x = quat[1],
                y = quat[2],
                z = quat[3],
                w = quat[4]
            }
        }

        -- Publicar
        simROS2.publish(pub, msg)

        -- Esperar un poco (como en el ejemplo oficial)
        sim.wait(0.05, false)
    end
end


function sysCall_cleanup()
    simROS2.shutdownPublisher(pub)
    sim.addLog(sim.verbosity_scriptinfos, "Publisher apagado")
end
```

Suscriptor de la velocidad comandada al robot

Código 5: Suscriptor de la velocidad comandada al robot.

```
function sysCall_init()
    sim = require'sim'
    simROS2 = require'simROS2'

    -- Obtener los motores del Khepera IV
    rightMotor=sim.getObjectHandle('K4_Right_Motor')
    leftMotor=sim.getObjectHandle('K4_Left_Motor')

    sim.setJointTargetVelocity(leftMotor, 0)
    sim.setJointTargetVelocity(rightMotor, 0)

    -- Parámetros físicos reales del Khepera IV
    wheelRadius = 0.021   -- metros (21 mm)
    wheelBase   = 0.088   -- metros (88 mm)

    -- Crear subscriber a /cmd_vel (Twist)
    sub = simROS2.createSubscription(
        '/cmd_vel',
        'geometry_msgs/msg/Twist',
        cmdVel_callback
    )

    sim.addLog(sim.verbosity_scriptinfos, "Subscriber /cmd_vel creado")
end


-- CALLBACK: recibe Twist y lo convierte a velocidades de rueda
function cmdVel_callback(msg)
    local v = msg.linear.x
    local w = msg.angular.z

    -- Cinemática diferencial
    local vLeft  = (v - w * wheelBase/2) / wheelRadius
    local vRight = (v + w * wheelBase/2) / wheelRadius

    -- Enviar velocidades a los motores
    sim.setJointTargetVelocity(leftMotor,  vLeft)
    sim.setJointTargetVelocity(rightMotor, vRight)
end


```

```
function sysCall_cleanup()
    simROS2.shutdownSubscription(sub)
    sim.addLog(sim.verbosity_scriptinfos, "Subscriber /cmd_vel apagado")
end
```

Servidor del servicio para obtener la posición del objeto que representa el punto de destino

Código 6: Servidor del servicio para obtener la posición del objeto que representa el punto de destino.

```
function sysCall_init()
    sim = require'sim'
    simROS2 = require'simROS2'

    -- Obtener el objeto TargetPosition
    target = sim.getObject('/TargetPosition')
end

function srv_callback(req)
    sim.addLog(sim.verbosity_msgs, 'service called')

    -- Obtener posición y orientación del target
    local pos = sim.getObjectPosition(target, -1)
    local quat = sim.getObjectQuaternion(target, -1)

    -- Crear respuesta Trigger
    local resp = simROS2.createInterface('std_srvs/srv/TriggerResponse')
    resp.success = true
    resp.message = string.format(
        '{"position":{"x":%f,"y":%f,"z":%f},"orientation":{"x":%f,"y":%f,"z":%f,"w":%f}}',
        pos[1], pos[2], pos[3],
        quat[1], quat[2], quat[3], quat[4]
    )

    return resp
end

function sysCall_thread()
    sim.addLog(sim.verbosity_msgs, 'thread started')

    -- Crear servicio dentro del thread (como el ejemplo oficial)
    srv = simROS2.createService(
        '/get_target_pose',
        'std_srvs/srv/Trigger',
        srv_callback
    )

    -- Mantener vivo el hilo
    while true do
        sim.step()
    end
end

function sysCall_cleanup()
    simROS2.shutdownService(srv)
    sim.addLog(sim.verbosity_msgs, 'thread finished')
end
```

Acción personalizada `ExecuteTrajectory`

Código 7: Acción personalizada `ExecuteTrajectory`.

```
# Goal
geometry_msgs/Pose target_pose
---
# Result
```

```
bool success
string message
---
# Feedback
float64 distance
float64 e_theta
float64 v
float64 w
float64 time_elapsed
string state
```

Llamada a la acción personalizada `ExecuteTrajectory`

Código 8: Llamada a la acción personalizada `ExecuteTrajectory`.

```
ros2 action send_goal /execute_trajectory interfaces/action/ExecuteTrajectory "{target_pose: {position: {x: .75, y
ros2 action send_goal /execute_trajectory interfaces/action/ExecuteTrajectory "{target_pose: {position: {x: .nan,
```

Programa principal

Código 9: Programa principal.

```
#include "rclcpp/rclcpp.hpp"
#include "tarea1/khepera_controller.hpp"

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);

    auto node = std::make_shared<ICCController>();

    // node->callTargetService(node);

    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

Cabeceras del nodo *ROS2* del controlador del robot

Código 10: Cabeceras del nodo *ROS2* del controlador del robot.

```
#pragma once

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"

#include "std_srvs/srv/trigger.hpp"
#include "geometry_msgs/msg/pose.hpp"
#include "geometry_msgs/msg/twist.hpp"

#include "tarea1/metrics_utils.hpp"
#include "interfaces/action/execute_trajectory.hpp"

class ICCController : public rclcpp::Node
{
public:
    using ExecuteTrajectory = interfaces::action::ExecuteTrajectory;
    using GoalHandleExecuteTrajectory = rclcpp_action::ServerGoalHandle<ExecuteTrajectory>;

    ICCController();

private:
    // -------------------------
    // ACTION SERVER CALLBACKS
```

```
    // -------------------------
    rclcpp_action::GoalResponse handleGoal(
        const rclcpp_action::GoalUUID &,
        std::shared_ptr<const ExecuteTrajectory::Goal>);

    rclcpp_action::CancelResponse handleCancel(
        const std::shared_ptr<GoalHandleExecuteTrajectory> goal_handle);

    void handleAccepted(
        const std::shared_ptr<GoalHandleExecuteTrajectory> goal_handle);

    // -------------------------
    // SERVICE RESPONSE CALLBACK (ASÍNCRONO)
    // -------------------------
    void onTargetReceived(
        const std_srvs::srv::Trigger::Response::SharedPtr response);

    // -------------------------
    // ROS CALLBACKS
    // -------------------------
    void robotPoseCallback(const geometry_msgs::msg::Pose::SharedPtr msg);
    void controlLoop();

    // -------------------------
    // ESTADO INTERNO
    // -------------------------
    double xt_, yt_;
    bool target_received_ = false;
    bool pose_received_   = false;

    geometry_msgs::msg::Pose robot_pose_;

    // -------------------------
    // ROS INTERFACES
    // -------------------------
    rclcpp::Client<std_srvs::srv::Trigger>::SharedPtr client_;
    rclcpp::Subscription<geometry_msgs::msg::Pose>::SharedPtr pose_sub_;
    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr cmd_pub_;
    rclcpp::TimerBase::SharedPtr timer_;

    // Action server
    rclcpp_action::Server<ExecuteTrajectory>::SharedPtr action_server_;
    std::shared_ptr<GoalHandleExecuteTrajectory> goal_handle_;

    // -------------------------
    // PARÁMETROS
    // -------------------------
    double kp_, vmax_, wmax_;
    int controller_type_;

    // -------------------------
    // MÉTRICAS
    // -------------------------
    std::unique_ptr<MetricsLogger> metrics_;
    double time_elapsed_ = 0.0;
};
```

Nodo ROS2 del controlador del robot

Código 11: Nodo ROS2 del controlador del robot.

```
#include <filesystem>
#include <cmath>

#include "ament_index_cpp/get_package_share_directory.hpp"

#include "tarea1/khepera_controller.hpp"
#include "tarea1/service_utils.hpp"
#include "tarea1/pose_utils.hpp"

using namespace std::chrono_literals;

```

```
ICCController::ICCController()
: Node("khepera_controller")
{
    // Parámetros
    this->declare_parameter("kp", 0.5);
    this->declare_parameter("vmax", 0.15);
    this->declare_parameter("wmax", 1.0);
    this->declare_parameter("controller_type", 0);

    kp_    = get_parameter("kp").as_double();
    vmax_  = get_parameter("vmax").as_double();
    wmax_  = get_parameter("wmax").as_double();
    controller_type_ = get_parameter("controller_type").as_int();

    // Nombre del controlador
    std::string controller_name =
        (controller_type_ == 0) ? "icc_reactivo" : "icc_geometrico";

    // Formateo de Kp
    std::string kp_str = std::to_string(kp_);
    kp_str.erase(kp_str.find_last_not_of('0') + 1);
    if (kp_str.back() == '.') kp_str.pop_back();

    // Ruta al share
    std::string pkg_share = ament_index_cpp::get_package_share_directory("tarea1");
    std::filesystem::path pkg_path = std::filesystem::path(pkg_share)
        .parent_path().parent_path().parent_path().parent_path();

    std::string results_dir = (pkg_path / "src" / "tarea1" / "results").string() + "/";
    std::filesystem::create_directories(results_dir);

    metrics_ = std::make_unique<MetricsLogger>(
        results_dir + controller_name + "_Kp" + kp_str,
        kp_, vmax_, wmax_, controller_type_
    );

    // Suscriptor
    pose_sub_ = create_subscription<geometry_msgs::msg::Pose>(
        "/robot_pose", 10,
        std::bind(&ICCController::robotPoseCallback, this, std::placeholders::_1));

    // Publicador
    cmd_pub_ = create_publisher<geometry_msgs::msg::Twist>("/cmd_vel", 10);

    // Cliente del servicio
    client_ = create_client<std_srvs::srv::Trigger>("/get_target_pose");

    // Action server
    action_server_ = rclcpp_action::create_server<ExecuteTrajectory>(
        this,
        "/execute_trajectory",
        std::bind(&ICCController::handleGoal, this, std::placeholders::_1, std::placeholders::_2),
        std::bind(&ICCController::handleCancel, this, std::placeholders::_1),
        std::bind(&ICCController::handleAccepted, this, std::placeholders::_1)
    );

    // Timer del control
    timer_ = create_wall_timer(
        50ms,
        std::bind(&ICCController::controlLoop, this));
}

// ---------------------------------------------------------------------------
// ACTION SERVER CALLBACKS
// ---------------------------------------------------------------------------

rclcpp_action::GoalResponse ICCController::handleGoal(
    const rclcpp_action::GoalUUID &,
    std::shared_ptr<const ExecuteTrajectory::Goal>)
{
    RCLCPP_INFO(get_logger(), "Nueva meta recibida: ejecutar trayectoria");
    return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
}

rclcpp_action::CancelResponse ICCController::handleCancel(
```

```
    const std::shared_ptr<GoalHandleExecuteTrajectory>)
{
    RCLCPP_INFO(get_logger(), "Cancelando trayectoria");
    return rclcpp_action::CancelResponse::ACCEPT;
}

void ICCController::handleAccepted(
    const std::shared_ptr<GoalHandleExecuteTrajectory> goal_handle)
{
    goal_handle_ = goal_handle;
    target_received_ = false;

    auto goal = goal_handle->get_goal();

    // Detectar si el target_pose está vacío (NaN)
    bool target_is_empty =
        std::isnan(goal->target_pose.position.x) ||
        std::isnan(goal->target_pose.position.y) ||
        std::isnan(goal->target_pose.position.z);

    if (target_is_empty)
    {
        RCLCPP_INFO(get_logger(), "No se recibió target → solicitándolo al servicio...");

        // Llamada ASÍNCRONA al servicio
        requestTargetPositionAsync(
            client_,
            std::bind(&ICCController::onTargetReceived, this, std::placeholders::_1),
            get_logger()
        );
    }
    else
    {
        RCLCPP_INFO(get_logger(), "Target recibido en la acción → usando target_pose directamente");

        // Guardar el target directamente (igual que hace onTargetReceived)
        xt_ = goal->target_pose.position.x;
        yt_ = goal->target_pose.position.y;

        target_received_ = true;

        // NO LLAMAR A executeControlLoop()
        // El timer controlLoop() se encargará automáticamente
    }
}



// ---------------------------------------------------------------------------
// CALLBACK DE RESPUESTA DEL SERVICIO (ASÍNCRONO)
// ---------------------------------------------------------------------------

void ICCController::onTargetReceived(
    const std_srvs::srv::Trigger::Response::SharedPtr response)
{
    if (!response) {
        RCLCPP_ERROR(get_logger(), "No se pudo obtener el target");
        auto result = std::make_shared<ExecuteTrajectory::Result>();
        result->success = false;
        result->message = "No se pudo obtener el target";
        goal_handle_->abort(result);
        return;
    }

    int parsed = sscanf(response->message.c_str(),
                        "{\"position\":{\"x\":%lf,\"y\":%lf",
                        &xt_, &yt_);

    if (parsed != 2) {
        RCLCPP_ERROR(get_logger(), "No se pudo parsear la respuesta del servicio.");
        auto result = std::make_shared<ExecuteTrajectory::Result>();
        result->success = false;
        result->message = "Respuesta inválida del servicio";
        goal_handle_->abort(result);
        return;
```

```
    }

    target_received_ = true;
    RCLCPP_INFO(get_logger(), "Objetivo recibido: x=%.3f y=%.3f", xt_, yt_);
}

// ---------------------------------------------------------------------------
// POSE CALLBACK
// ---------------------------------------------------------------------------

void ICCController::robotPoseCallback(const geometry_msgs::msg::Pose::SharedPtr msg)
{
    robot_pose_ = *msg;
    pose_received_ = true;
}

// ---------------------------------------------------------------------------
// CONTROL LOOP
// ---------------------------------------------------------------------------

void ICCController::controlLoop()
{
    if (!goal_handle_) return;
    if (!pose_received_) return;
    if (!target_received_) return;

    auto feedback = std::make_shared<ExecuteTrajectory::Feedback>();

    time_elapsed_ += 0.05;

    double xr, yr, theta_r;
    extractPose(robot_pose_, xr, yr, theta_r);

    double dx = xt_ - xr;
    double dy = yt_ - yr;
    double d = std::sqrt(dx*dx + dy*dy);

    double theta_d = std::atan2(dy, dx);
    double e_theta = normalizeAngle(theta_d - theta_r);

    double v = kp_ * d;
    if (v > vmax_) v = vmax_;

    double w = 0.0;
    if (controller_type_ == 0)
        w = wmax_ * std::sin(e_theta);
    else {
        double R = d / (2 * std::sin(e_theta));
        w = v / R;
    }

    // Feedback
    feedback->distance = d;
    feedback->e_theta = e_theta;
    feedback->v = v;
    feedback->w = w;
    feedback->time_elapsed = time_elapsed_;

    goal_handle_->publish_feedback(feedback);

    // Registrar métricas acumuladas
    metrics_->updateMetrics(d, time_elapsed_);

    // Registrar TODA la información de esta iteración
    metrics_->addSample(
        time_elapsed_,
        xr, yr, theta_r,   // pose del robot
        xt_, yt_,          // objetivo
        d, e_theta,        // errores
        v, w               // señales de control
    );

    // Llegada al objetivo
    if (d < 0.05) {
        RCLCPP_INFO(get_logger(), "Objetivo alcanzado");
```

```
        cmd_pub_->publish(geometry_msgs::msg::Twist());

        metrics_->saveToCSV();

        auto result = std::make_shared<ExecuteTrajectory::Result>();
        result->success = true;
        result->message = "Objetivo alcanzado";

        goal_handle_->succeed(result);
        goal_handle_.reset();
        return;
    }

    // Publicar comando
    geometry_msgs::msg::Twist cmd;
    cmd.linear.x = v;
    cmd.angular.z = w;
    cmd_pub_->publish(cmd);
}
```

Cabeceras del módulo encargado de la llamada asíncrona al servicio /get_target_pose

Código 12: Cabeceras del módulo encargado de la llamada asíncrona al servicio /get_target_pose.

```
#pragma once

#include "rclcpp/rclcpp.hpp"
#include "std_srvs/srv/trigger.hpp"

// Lanza la petición asíncrona.
// El callback se ejecutará cuando llegue la respuesta.
void requestTargetPositionAsync(
    rclcpp::Client<std_srvs::srv::Trigger>::SharedPtr client,
    std::function<void(const std_srvs::srv::Trigger::Response::SharedPtr)> callback,
    rclcpp::Logger logger);
```

Módulo encargado de la llamada asíncrona al servicio /get_target_pose

Código 13: Módulo encargado de la llamada asíncrona al servicio /get_target_pose.

```
#include "tarea1/service_utils.hpp"

void requestTargetPositionAsync(
    rclcpp::Client<std_srvs::srv::Trigger>::SharedPtr client,
    std::function<void(const std_srvs::srv::Trigger::Response::SharedPtr)> callback,
    rclcpp::Logger logger)
{
    using namespace std::chrono_literals;

    RCLCPP_INFO(logger, "Esperando servicio /get_target_pose...");

    if (!client->wait_for_service(1s)) {
        RCLCPP_ERROR(logger, "Servicio no disponible.");
        callback(nullptr);
        return;
    }

    auto request = std::make_shared<std_srvs::srv::Trigger::Request>();

    // Petición asíncrona
    client->async_send_request(
        request,
        [callback, logger](rclcpp::Client<std_srvs::srv::Trigger>::SharedFuture future)
        {
```

```
            try {
                auto response = future.get();
                callback(response);
            }
            catch (...) {
                RCLCPP_ERROR(logger, "Error al recibir respuesta del servicio.");
                callback(nullptr);
            }
        }
    );
}
```

Cabeceras del módulo encargado de extraer la posición

Código 14: Cabeceras del módulo encargado de extraer la posición.

```
#pragma once

#include "geometry_msgs/msg/pose.hpp"
#include <cmath>

inline void extractPose(
    const geometry_msgs::msg::Pose &pose,
    double &x, double &y, double &yaw)
{
    x = pose.position.x;
    y = pose.position.y;

    double qw = pose.orientation.w;
    double qx = pose.orientation.x;
    double qy = pose.orientation.y;
    double qz = pose.orientation.z;

    yaw = std::atan2(2*(qw*qz + qx*qy),
                     1 - 2*(qy*qy + qz*qz));
}

inline double normalizeAngle(double a)
{
    return std::atan2(std::sin(a), std::cos(a));
}
```

Cabeceras del módulo para calcular las métricas de evaluación

Código 15: Cabeceras del módulo para calcular las métricas de evaluación

```
#pragma once

#include <string>
#include <vector>

struct MetricsSample
{
    double time;
    double xr, yr, theta_r;
    double xt, yt;
    double d;
    double e_theta;
    double v;
    double w;
};

class MetricsLogger
{
public:
    MetricsLogger(const std::string &filename,
              double kp, double vmax, double wmax, int controller_type);

    void updateMetrics(double error, double time);
```

```
    void addSample(double time,
                   double xr, double yr, double theta_r,
                   double xt, double yt,
                   double d, double e_theta,
                   double v, double w);

    void saveToCSV();

private:
    double iae_  = 0.0;
    double ise_  = 0.0;
    double itae_ = 0.0;
    double itse_ = 0.0;

    double dt_ = 0.05;

    std::string filename_;

    // Configuración del controlador
    double kp_, vmax_, wmax_;
    int controller_type_;

    std::vector<MetricsSample> samples_;
};
```

Módulo para calcular las métricas de evaluación

Código 16: Módulo para calcular las métricas de evaluación

```
#include "tarea1/metrics_utils.hpp"
#include <fstream>
#include <cmath>

MetricsLogger::MetricsLogger(const std::string &filename,
                             double kp, double vmax, double wmax, int controller_type)
    : kp_(kp), vmax_(vmax), wmax_(wmax), controller_type_(controller_type)
{
    filename_ = filename + ".csv";
}

void MetricsLogger::updateMetrics(double error, double time)
{
    iae_  += std::abs(error) * dt_;
    ise_  += error * error * dt_;
    itae_ += time * std::abs(error) * dt_;
    itse_ += time * error * error * dt_;
}

void MetricsLogger::addSample(double time,
                              double xr, double yr, double theta_r,
                              double xt, double yt,
                              double d, double e_theta,
                              double v, double w)
{
    samples_.push_back({time, xr, yr, theta_r, xt, yt, d, e_theta, v, w});
}

void MetricsLogger::saveToCSV()
{
    std::ofstream file(filename_);

    // Cabecera
    file << "time,xr,yr,theta_r,xt,yt,d,e_theta,v,w\n";

    // Datos por iteración
    for (const auto &s : samples_) {
        file << s.time << ","
             << s.xr << "," << s.yr << "," << s.theta_r << ","
             << s.xt << "," << s.yt << ","
             << s.d << "," << s.e_theta << ","
             << s.v << "," << s.w << "\n";
    }
```

```

    // Métricas finales
    file << "\nMetric,Value\n";
    file << "IAE,"  << iae_  << "\n";
    file << "ISE,"  << ise_  << "\n";
    file << "ITAE," << itae_ << "\n";
    file << "ITSE," << itse_ << "\n";

    // Configuración del controlador
    file << "\nParameter,Value\n";
    file << "kp,"   << kp_   << "\n";
    file << "vmax," << vmax_ << "\n";
    file << "wmax," << wmax_ << "\n";
    file << "controller_type," << controller_type_ << "\n";

    file.close();
}
```